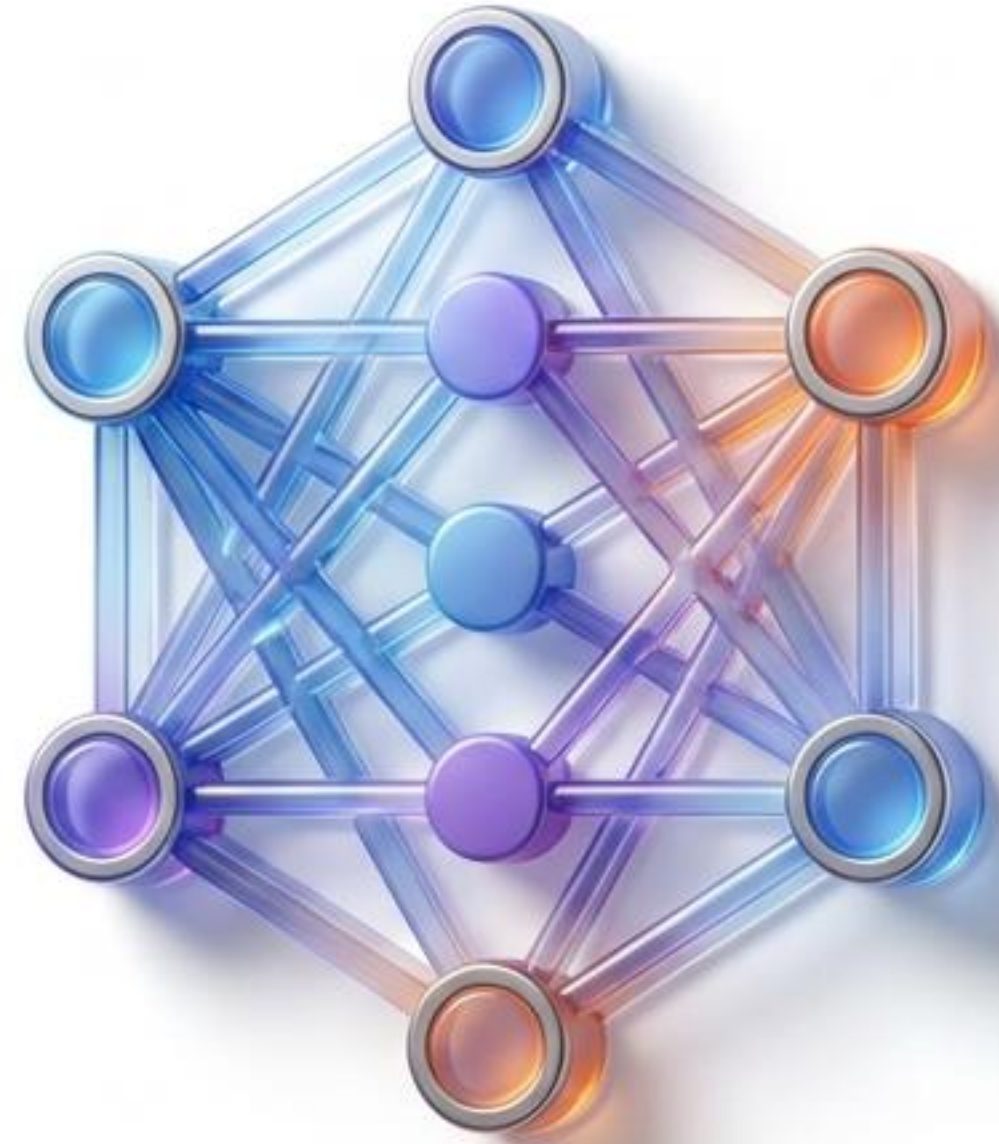


Học Máy (Machine Learning)

Chương 8: Giảm chiều dữ liệu (Dimensionality Reduction)

Giảng viên: TS. Trần Thành Thắng
Đại học Đông Á
August 5, 2026



Nội dung Chương trình



**Khái quát về
Khái quát về Giảm
chiều dữ liệu**



**Phân tích thành
phần chính (PCA)**



**Phép chiếu ngẫu
Phép chiếu ngẫu
nhiên & LLE**

Giới thiệu Giảm chiều dữ liệu

Trong thực tế, nhiều bài toán học máy liên quan đến hàng nghìn hoặc hàng triệu đặc trưng (features) cho mỗi trường hợp huấn luyện. Việc này không chỉ làm quá trình huấn luyện cực kỳ chậm mà còn làm cho việc tìm ra một giải pháp tốt trở nên khó khăn hơn.

Vấn đề này thường được gọi là lời nguyền của số chiều (the curse of dimensionality).



Kỹ thuật **giảm chiều** (Dimensionality Reduction) giúp biến bài toán phức tạp thành bài toán có thể giải quyết được bằng cách loại bỏ các đặc trưng không quan trọng hoặc dư thừa.



Lợi ích 1: Tăng tốc quá trình huấn luyện

Giảm số lượng đặc trưng giúp thuật toán học máy chạy nhanh hơn đáng kể, tốn ít bộ nhớ hơn.



giúp thuật toán học máy chạy nhanh hơn đáng kể, tốt liền.

Ví dụ (Ảnh MNIST)

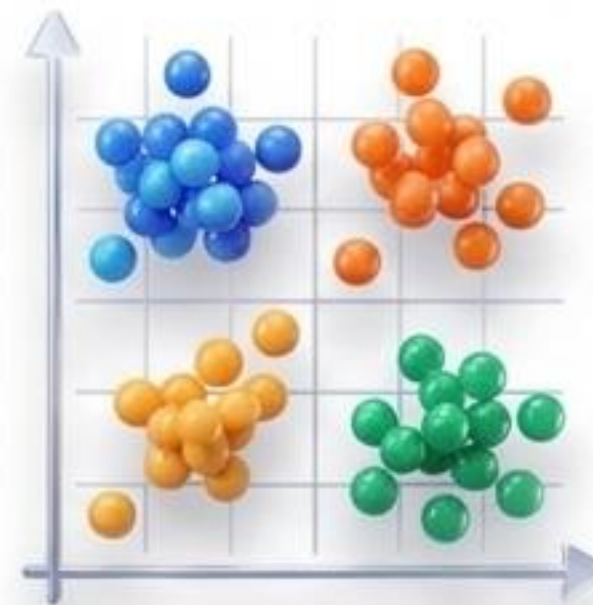


Các pixel ở đường biên hình ảnh hầu như luôn trắng. Ta có thể loại bỏ hoàn toàn các pixel này khỏi tập huấn luyện mà không làm mất nhiều thông tin hữu ích cho tác vụ phân loại chữ số.

Hai pixel lân cận thường có tương quan cao, có thể hợp nhất chúng (ví dụ: lấy trung bình) để giảm dữ liệu.

Lợi ích 2: Trực quan hóa dữ liệu

Trực quan hóa là điều cần thiết để truyền đạt kết luận của nhà khoa học dữ liệu cho những người ra quyết định. Rất khó để vẽ một đồ thị có hơn 3 chiều.



Giảm số lượng chiều xuống còn 2 hoặc 3 giúp vẽ biểu đồ trực quan, giúp con người dễ dàng quan sát và phát hiện các mẫu (chẳng hạn như các cụm/clusters).

Lời nguyên của số chiều

Chúng ta quen sống trong không gian 3D, do đó trực giác thường thất bại khi hình dung không gian có chiều cao (ví dụ: 1.000 chiều). Trong không gian chiều cao, nhiều thứ hoạt động rất khác.



2D

0.4%

Một điểm ngẫu nhiên trong hình vuông 1x1 (2D) chỉ có 0.4% cơ hội nằm cách đường biên dưới 0.001.



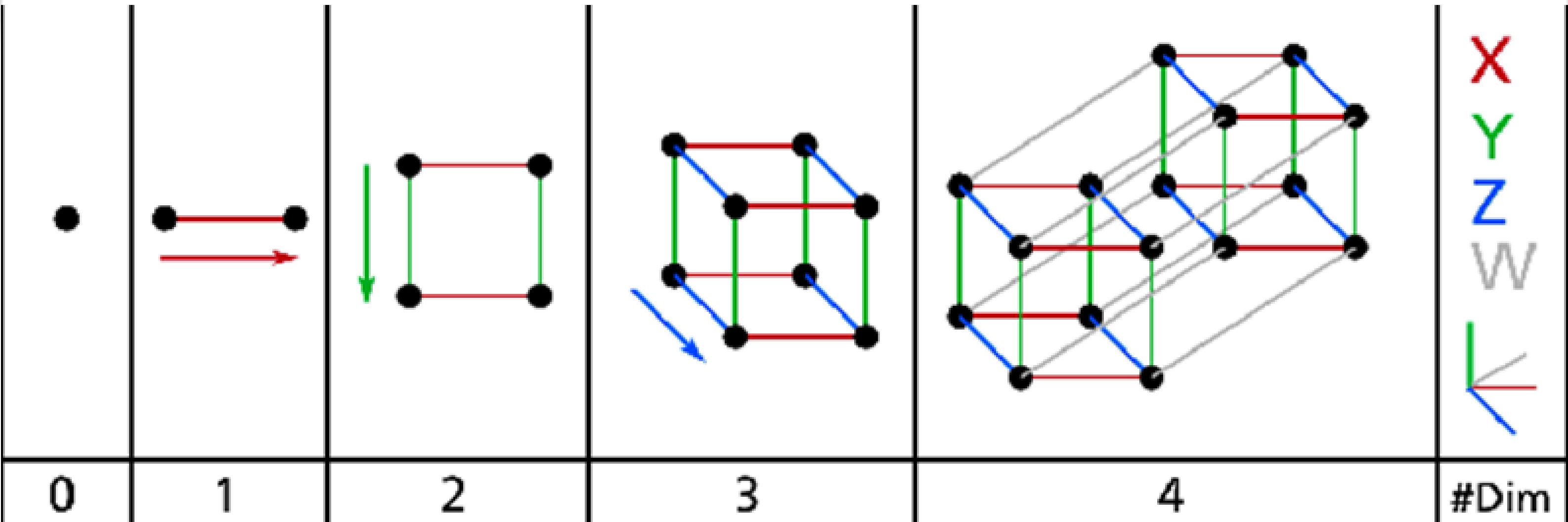
10,000D

> 99.9999999%

Trong một siêu hình lập phương 10.000 chiều, xác suất này > 99.9999999%.

Lời nguyên của số chiều (Tiếp theo)

Kết luận: Hầu hết các điểm trong một không gian có chiều cao đều nằm rất gần đường biên.



Hình 8-1. Điểm, đoạn thẳng, hình vuông, hình lập phương và tesseract (siêu hình lập

Sự khác biệt khoảng cách trong không gian chiều cao



≈ 0.52

Nếu chọn hai điểm ngẫu nhiên trong một hình vuông 2D đơn vị, khoảng cách trung bình là



≈ 0.66

Trong hình lập phương 3D đơn vị, khoảng cách trung bình là



≈ 408.25

Trong một siêu hình lập phương đơn vị 1.000.000 chiều, khoảng cách trung bình là

Có rất nhiều không gian trong các chiều cao, dẫn đến dữ liệu vô cùng thưa thớt (*sparse*).

Nguy cơ quá khớp do thừa thớt

Tập dữ liệu có chiều cao $\rightarrow$ hầu hết các trường hợp huấn luyện cách rất xa nhau.

Một trường hợp mới (test instance) có thể sẽ cách xa bất kỳ trường hợp huấn luyện nào đã biết.

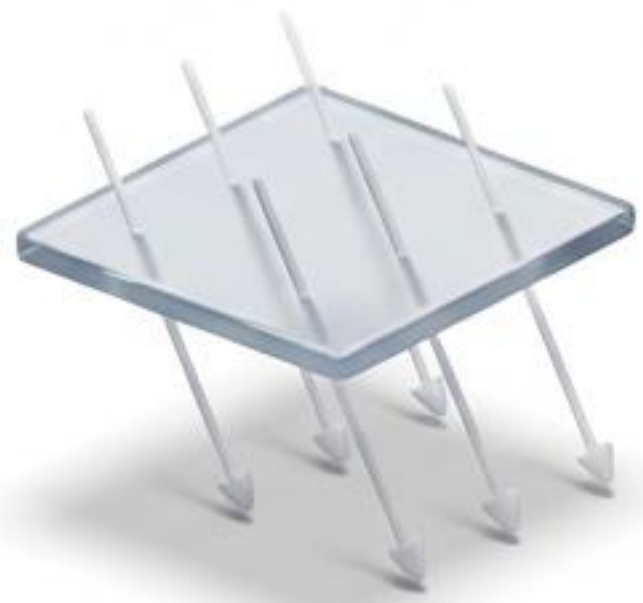
Dự đoán sẽ kém tin cậy hơn nhiều vì phải dựa trên các phép ngoại suy (extrapolation) lớn.

Tóm lại: Tập huấn luyện càng có nhiều chiều, nguy cơ **quá khớp (overfitting)** càng lớn.



Các cách tiếp cận chính để giảm chiều

Thay vì cố gắng thu thập dữ liệu bằng số nguyên tử vũ trụ (để đủ độ dày đặc), chúng ta dùng các thuật toán giảm chiều:



1. Phép chiếu (Projection)



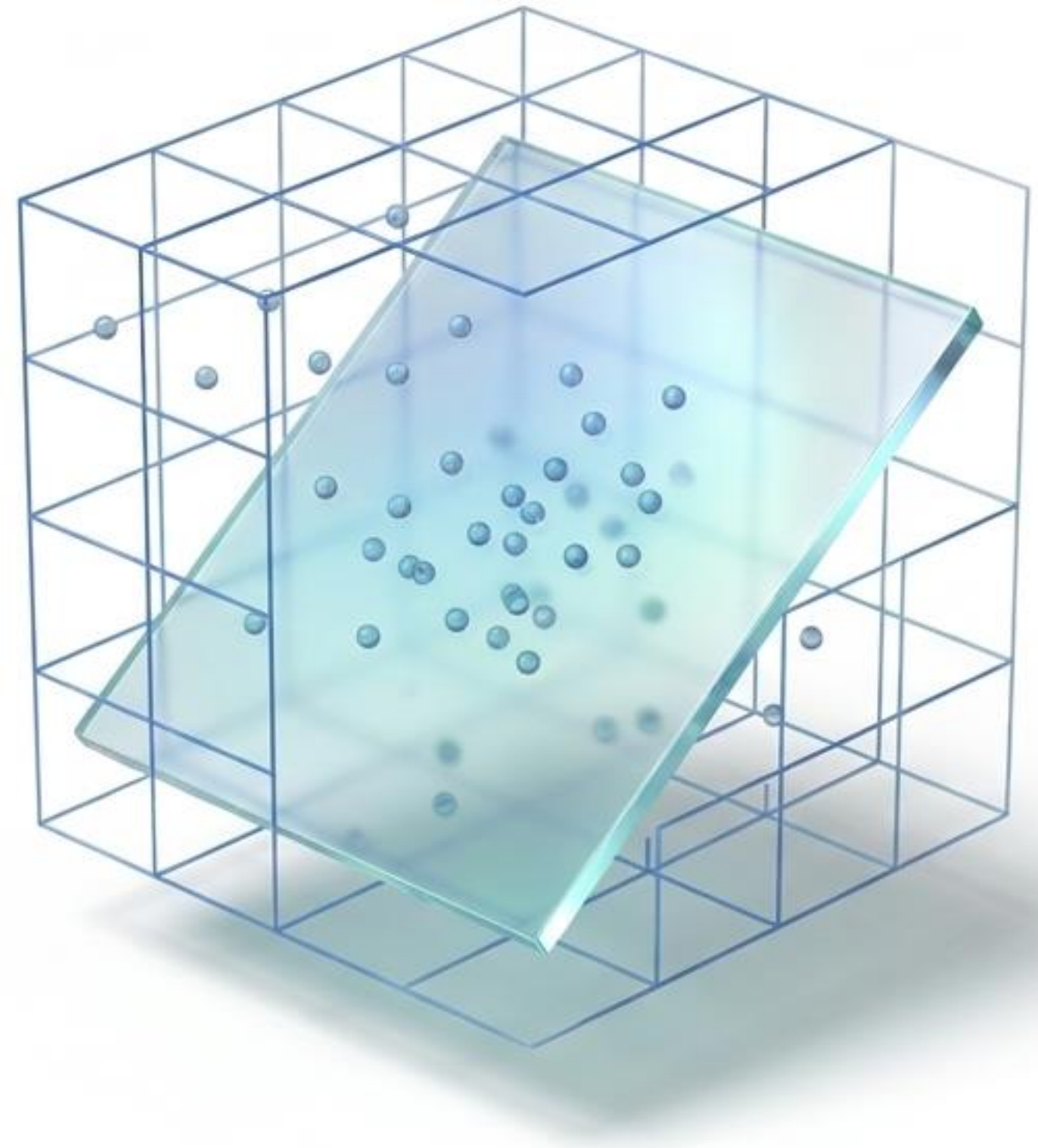
2. Học đa tạp (Manifold Learning)

Cách tiếp cận 1: Phép chiếu (Projection)

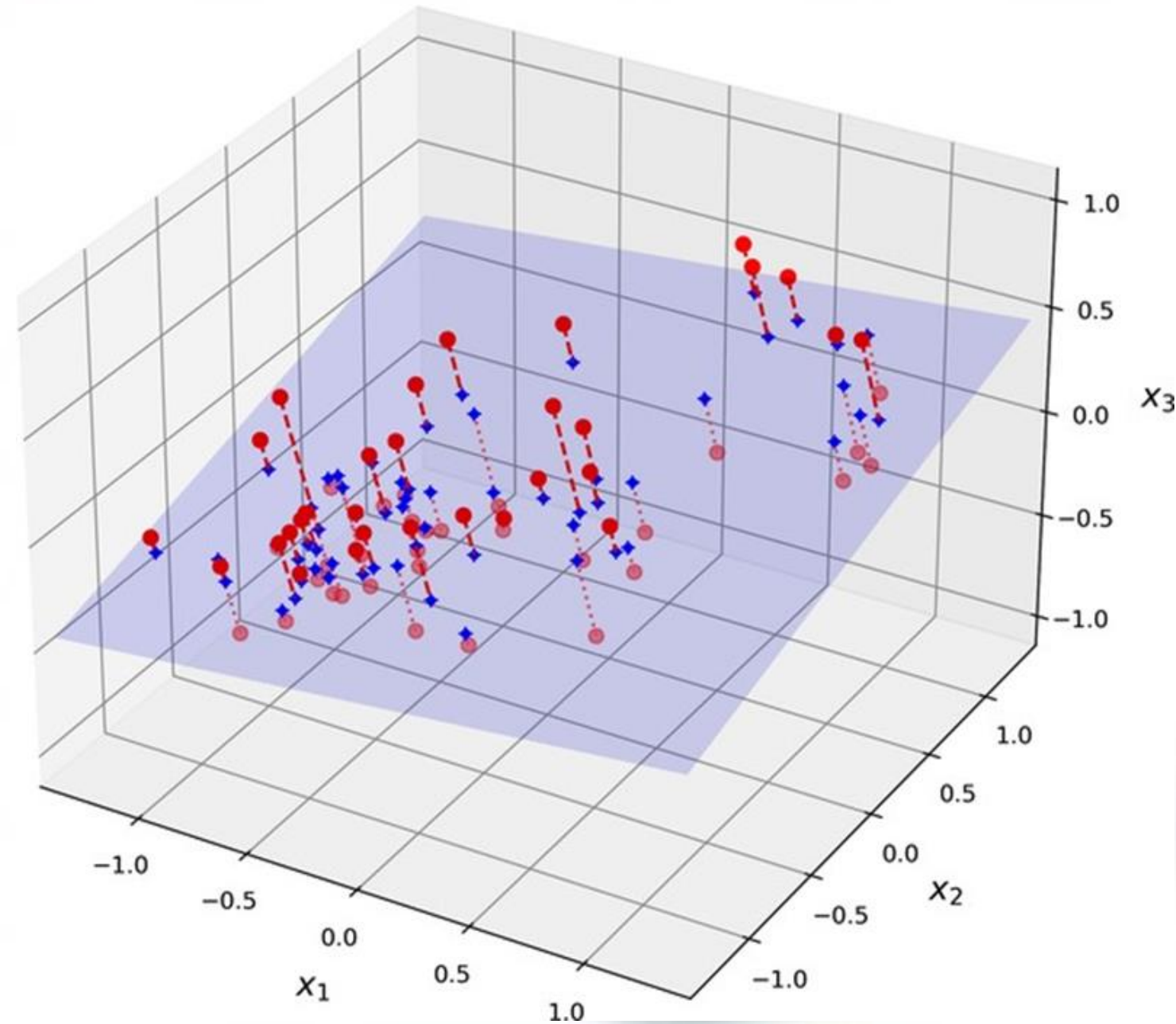
Trong hầu hết các bài toán thực tế, các trường hợp huấn luyện không được phân bố đều đặn (uniform) trên tất cả các chiều.

Nhiều đặc trưng gần như không đổi, trong khi các đặc trưng khác tương quan mạnh.

Kết quả là, mọi trường hợp huấn luyện thường nằm gọn trong (hoặc rất gần) một không gian con (subspace) có số chiều thấp hơn nhiều so với không gian gốc.

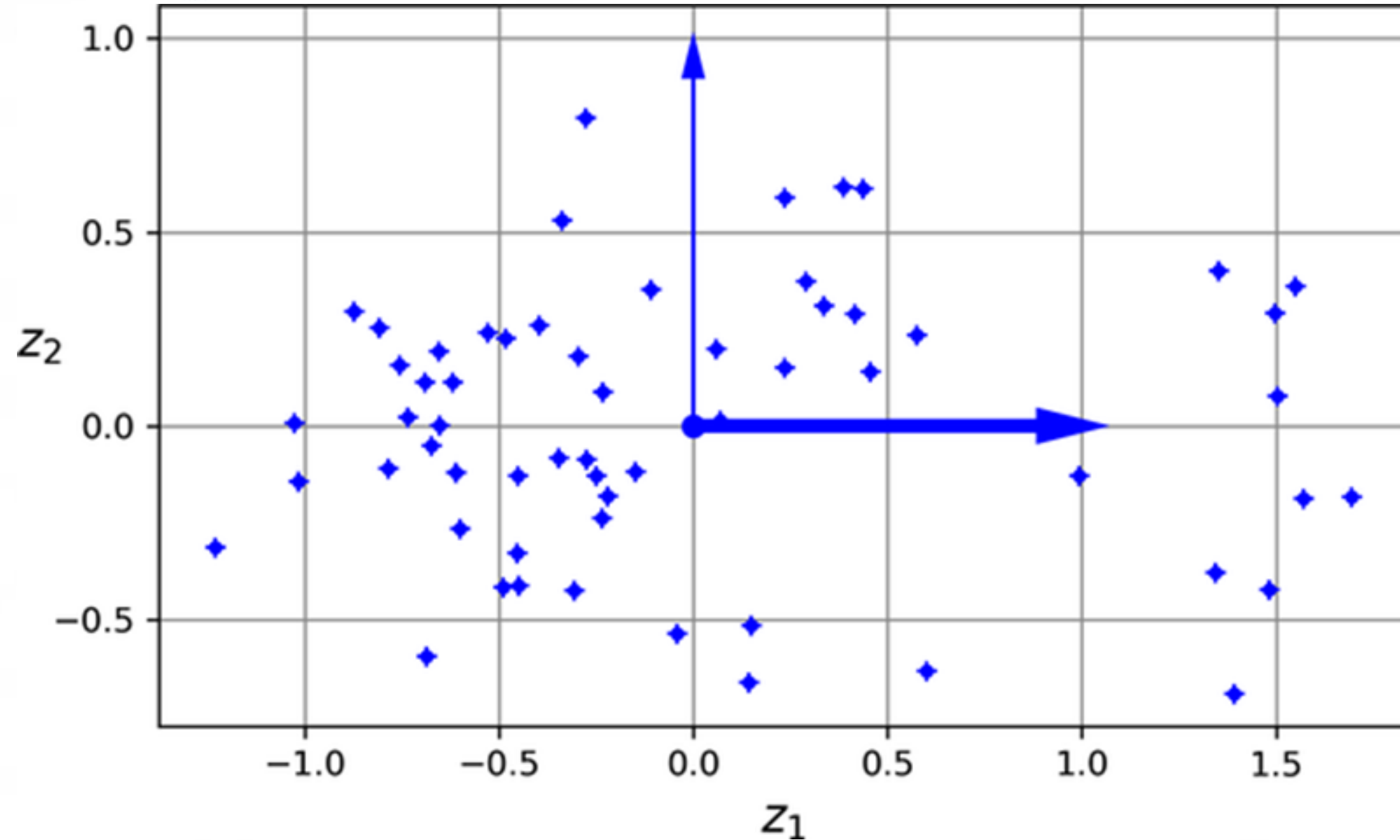


Minh họa Phép chiếu



Tập dữ liệu 3D gần không gian con 2D
Hình 8-2. Một tập dữ liệu 3D nằm gần một không gian con 2D (mặt phẳng)

Kết quả Phép chiếu



Tập dữ liệu 2D mới sau phép chiếu

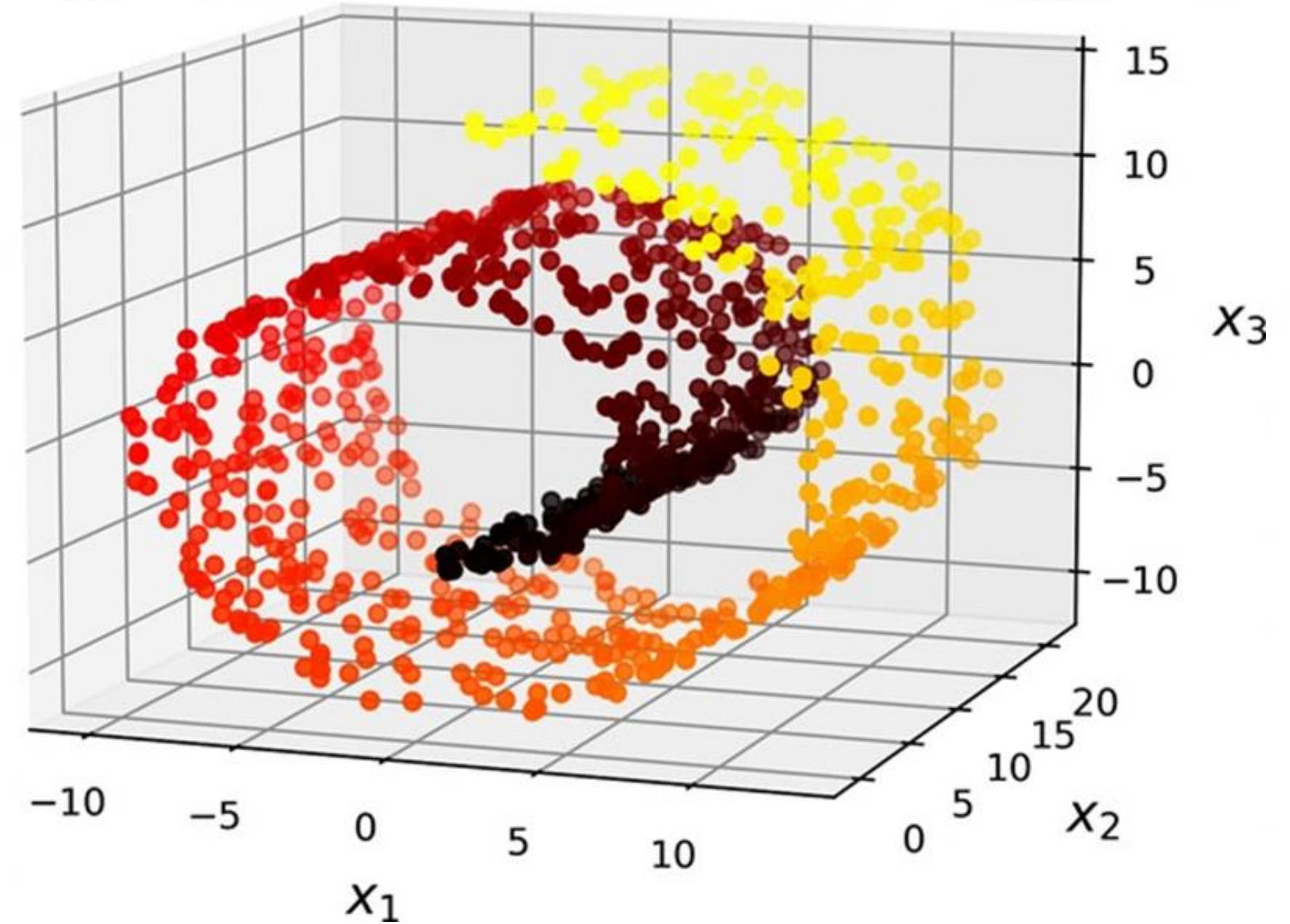
Hình 8-3. Bằng cách chiếu vuông góc các điểm lên mặt phẳng, ta thu được tập dữ liệu 2D mới

Cách tiếp cận 2: Học đa tạp (Manifold Learning)

Phép chiếu không phải lúc nào cũng là cách tiếp cận tốt nhất.

Trong nhiều trường hợp, không gian con có thể bị xoắn và uốn cong.

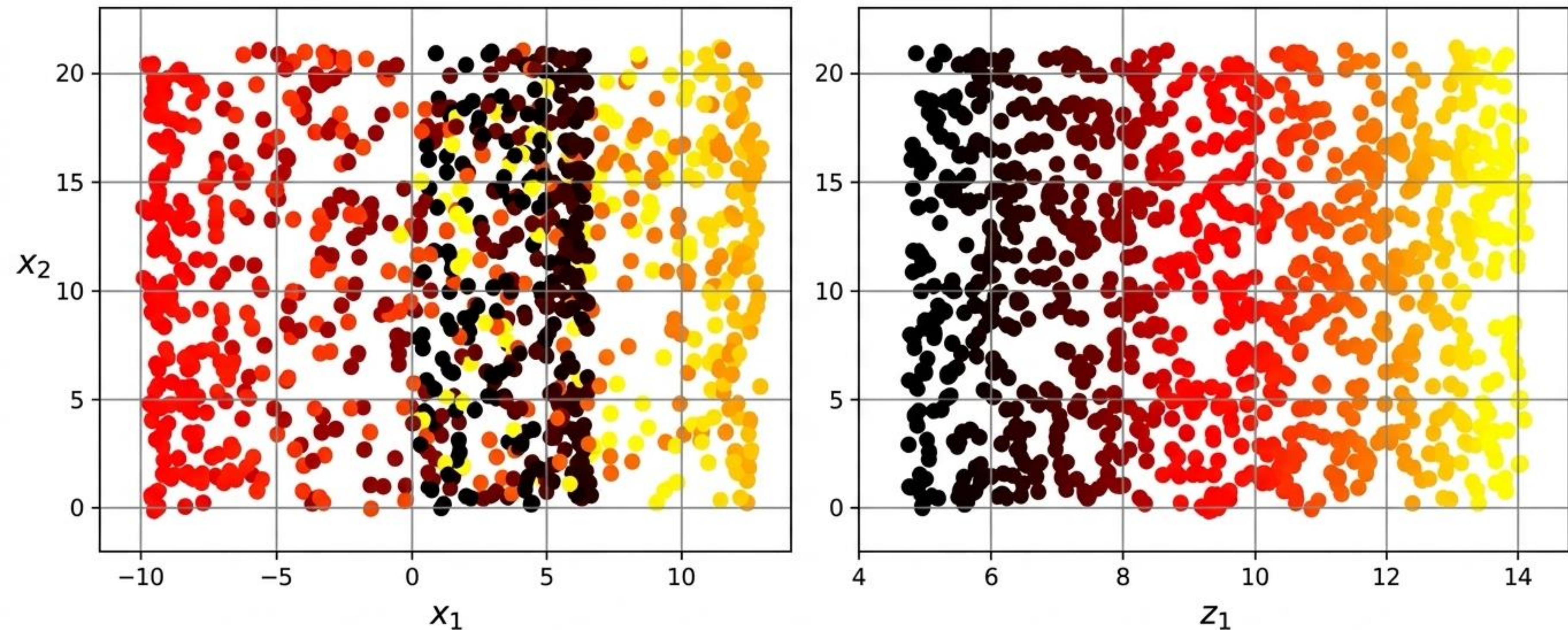
Ví dụ điển hình: Tập dữ liệu Swiss roll.



Tập dữ liệu Swiss roll

Hình 8-4. Tập dữ liệu Swiss roll (như một tờ giấy bị cuộn lại trong không gian 3D)

Làm dẹt (Phép chiếu) vs Mở cuộn (Đa tạp)



Hình 8-5. Chiếu vuông góc làm dẹt cuộn (trái) - sai lầm; Mở cuộn (phải) - chính xác

Giả định đa tạp (Manifold Hypothesis)



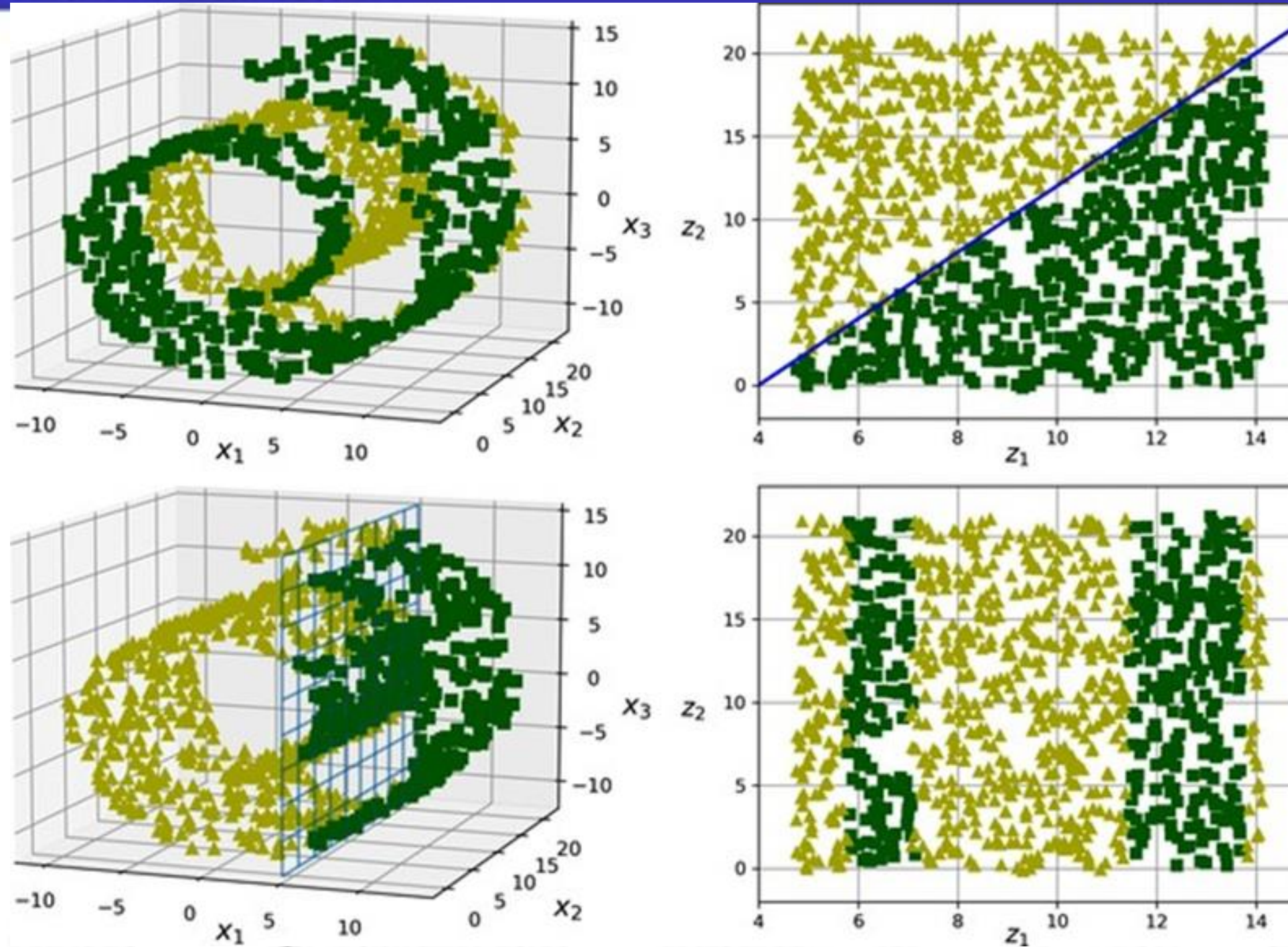
Đa tạp 2D: Là một hình 2D có thể bị uốn cong, xoắn lại trong một không gian có chiều cao hơn. Tổng quát, một đa tạp d -chiều nằm trong không gian n -chiều ($d < n$).



Giả định đa tạp: Hầu hết các tập dữ liệu thực tế có chiều cao đều nằm gần một đa tạp có chiều thấp hơn nhiều.

Các tác vụ học máy (như phân loại/hồi quy) thường sẽ đơn giản hơn nếu được biểu diễn trong không gian đa tạp (sau khi được mở cuộn).

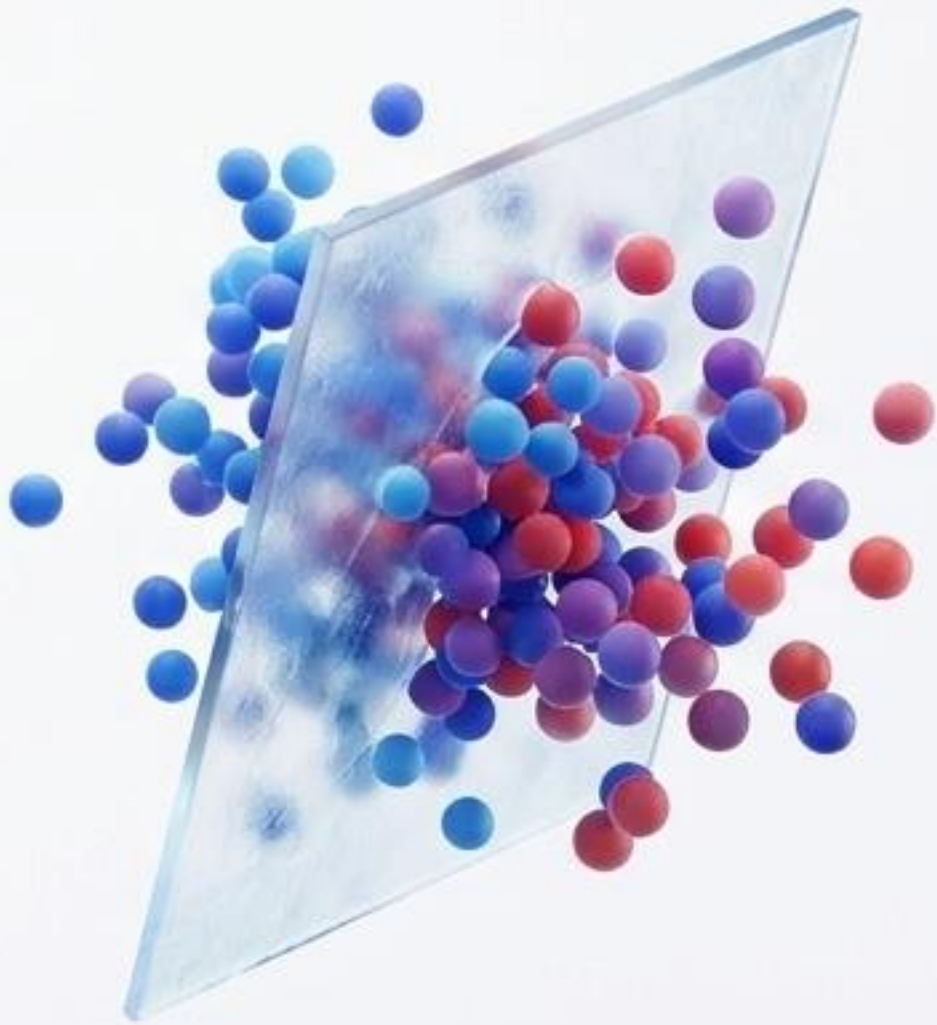
Lưu ý về Giả định đa tạp



Hình 8-6. Đường biên quyết định không phải lúc nào cũng đơn giản hơn với không gian chiều thấp

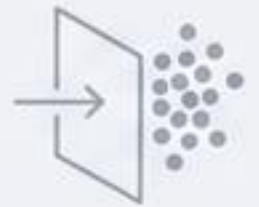
Phân tích thành phần chính (PCA)

Phân tích thành phần chính (Principal Component Analysis - PCA) cho đến nay là thuật toán giảm chiều phổ biến nhất. Nó là một phương pháp thuộc nhóm Phép chiếu (Projection).



1

Đầu tiên xác định siêu phẳng (hyperplane) nằm gần dữ liệu nhất.



2

Sau đó chiếu toàn bộ dữ liệu lên mặt phẳng đó.



Bảo toàn phương sai (Preserving Variance)

MỤC TIÊU

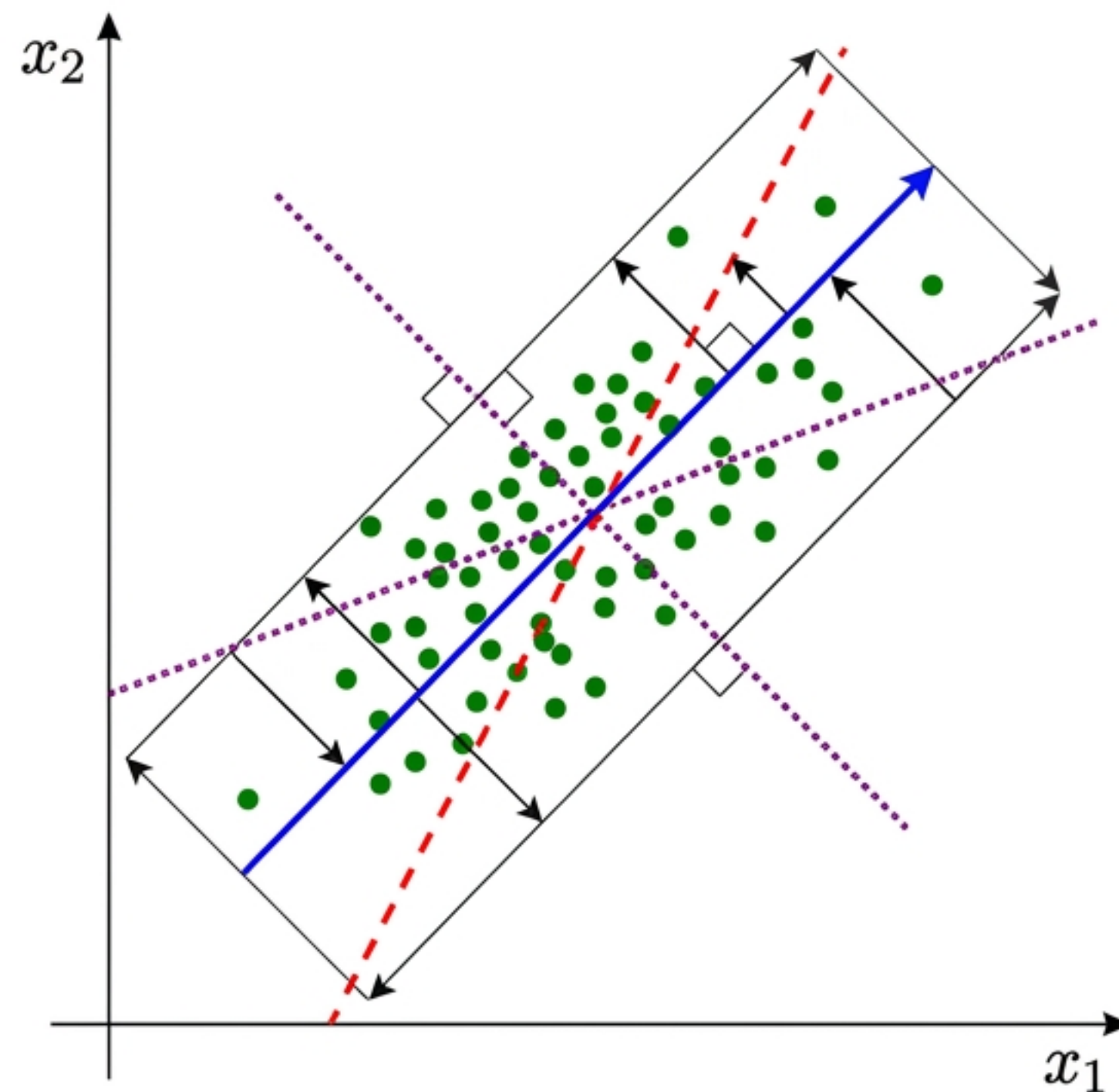
- Trước khi chiếu dữ liệu, cần phải chọn một siêu phẳng phù hợp.

NGUYÊN TẮC

- PCA sẽ chọn siêu phẳng (hoặc trục) sao cho bảo toàn được lượng phương sai lớn nhất của dữ liệu. Giúp giữ lại nhiều thông tin nhất có thể.

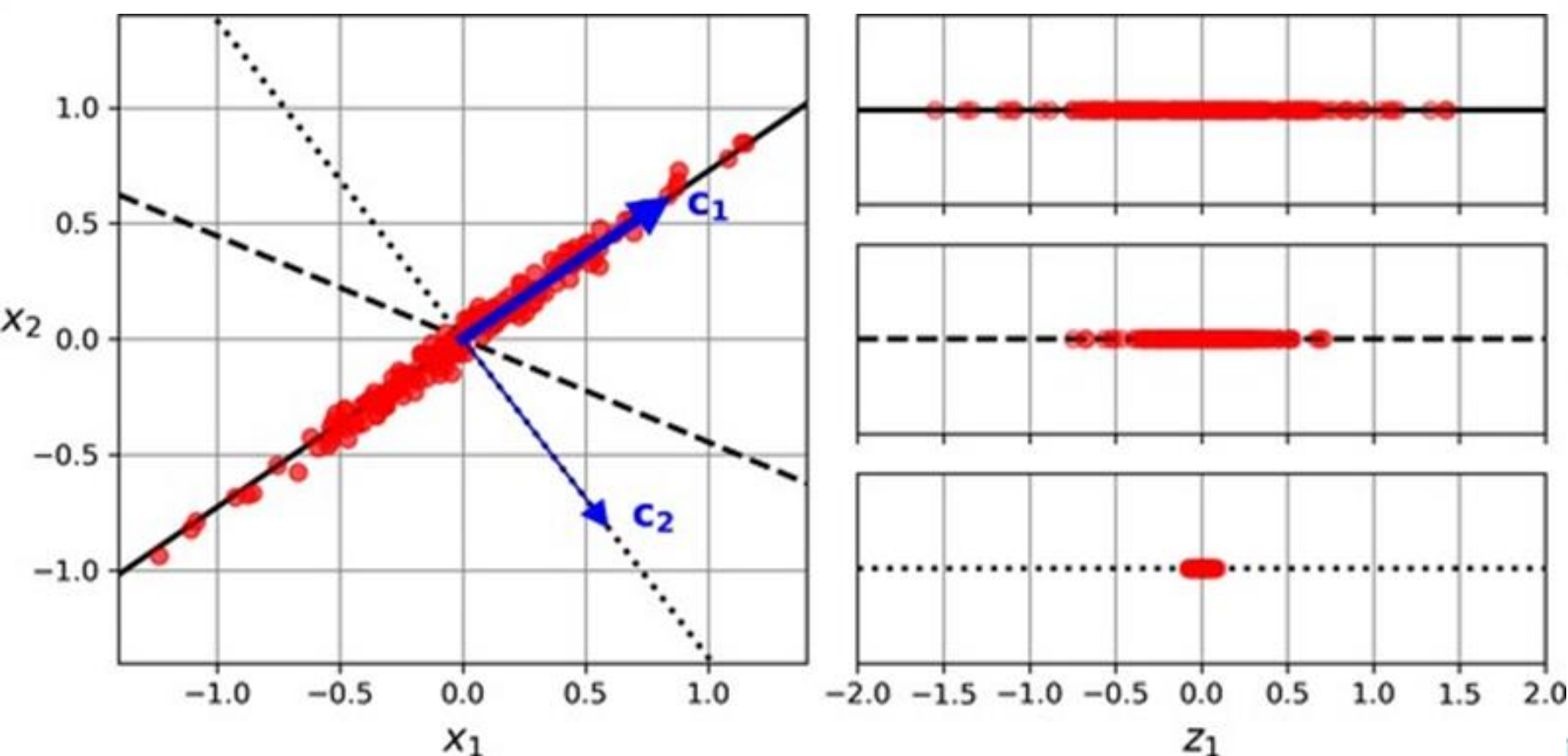
TÍNH TƯƠNG ĐƯƠNG

- Việc này cũng tương đương với việc chọn trục làm giảm thiểu khoảng cách bình phương trung bình giữa tập dữ liệu gốc và điểm chiếu.



Các thành phần chính (Principal Components - PC)

Lựa chọn không gian con để chiếu (Hình 8-7. Phép chiếu lên đường liền nét bảo toàn phương sai tối đa).



- 1 **PC1:** Tìm trục có phương sai lớn nhất trong tập huấn luyện.
- 2 **PC2:** Tìm trục thứ 2 trực giao (vuông góc) với trục 1 và chứa nhiều phương sai nhất từ phần còn lại.
- 3 **PC_n:** Quá trình tiếp diễn tối đa bằng số chiều của không gian gốc.

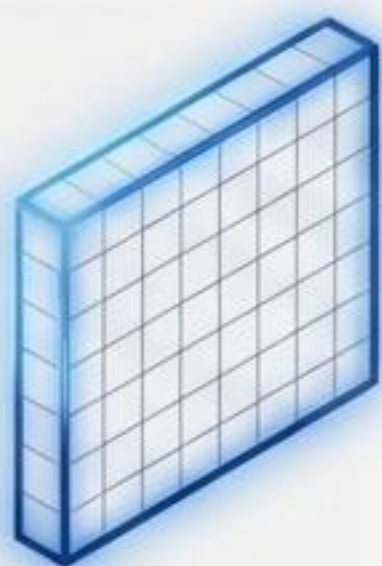
Trục thứ i được gọi là **thành phần chính thứ i** của dữ liệu.

Toán học PCA: Phân tách giá trị số ít (SVD)

Để tìm tất cả các thành phần chính, ta sử dụng kỹ thuật phân tách ma trận tiêu chuẩn gọi là **Phân tách giá trị số ít** (Singular Value Decomposition - SVD).

SVD phân tách ma trận huấn luyện $\mathbf{X}$ thành ba ma trận:

$$\mathbf{X} = \mathbf{U} \cdot \mathbf{\Sigma} \cdot \mathbf{V}^T$$



Ma trận $\mathbf{V}^T$ chứa tất cả các thành phần chính cần tìm (các vector đơn vị trực giao).

Công thức $\mathbf{V}^T$:
$$\mathbf{V}^T = \left(\begin{array}{c|c|c|c} | & | & \cdots & | \\ \mathbf{c}_1 & \mathbf{c}_2 & & \mathbf{c}_n \\ | & | & & | \end{array} \right)^T$$

Tính toán PCA bằng NumPy (Phần 1)

LOGIC



Dùng hàm `svd()` của NumPy. Lưu ý **quan trọng**: PCA giả định rằng dữ liệu được căn giữa (**centered**) xung quanh gốc tọa độ. Cần trừ đi trung bình trước khi chạy SVD.

SYNTAX

```
import numpy as np

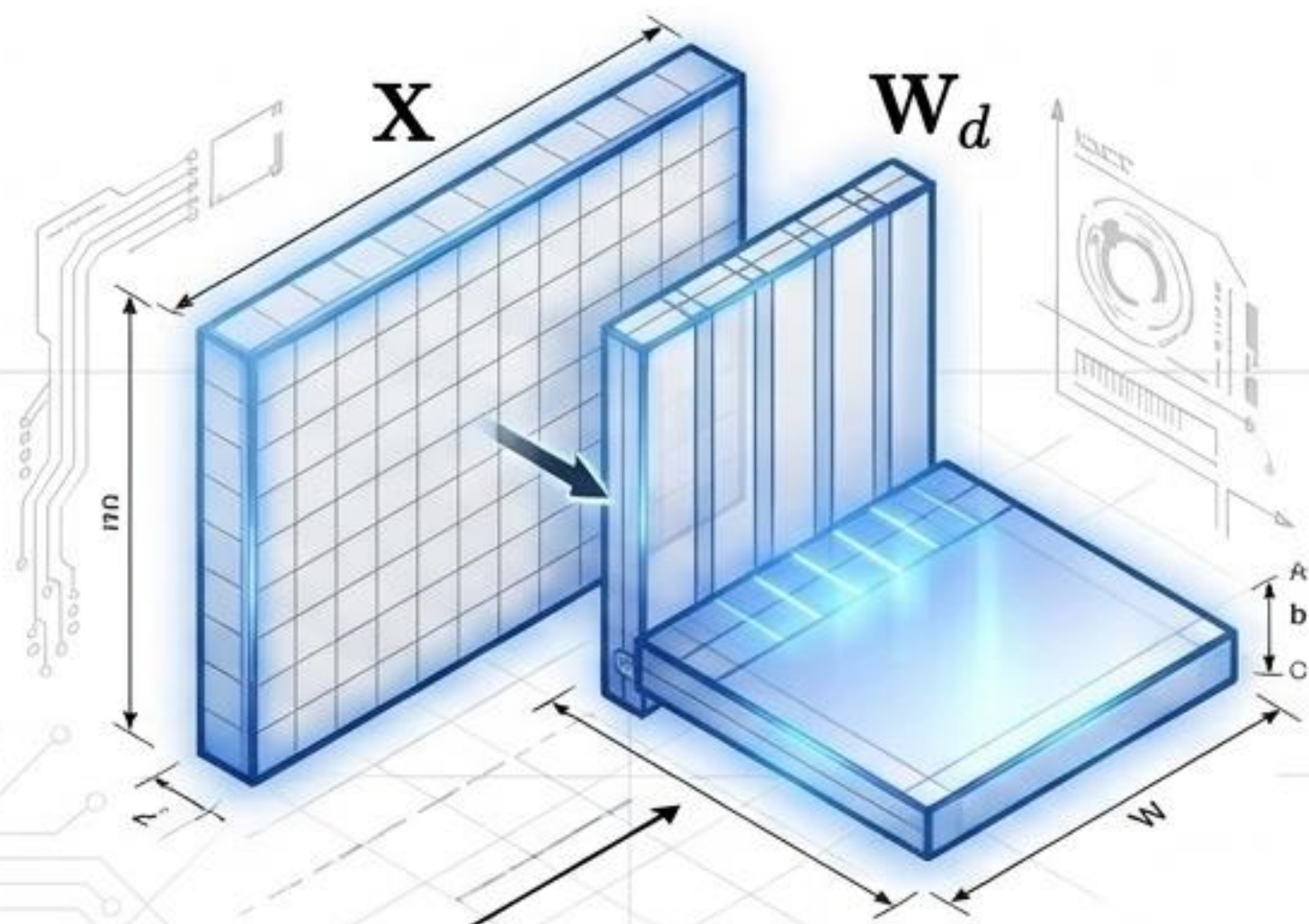
# Giả sử X là tập dữ liệu của bạn
X_centered = X - X.mean(axis=0)

# Phân tách giá trị số ít
U, s, Vt = np.linalg.svd(X_centered)

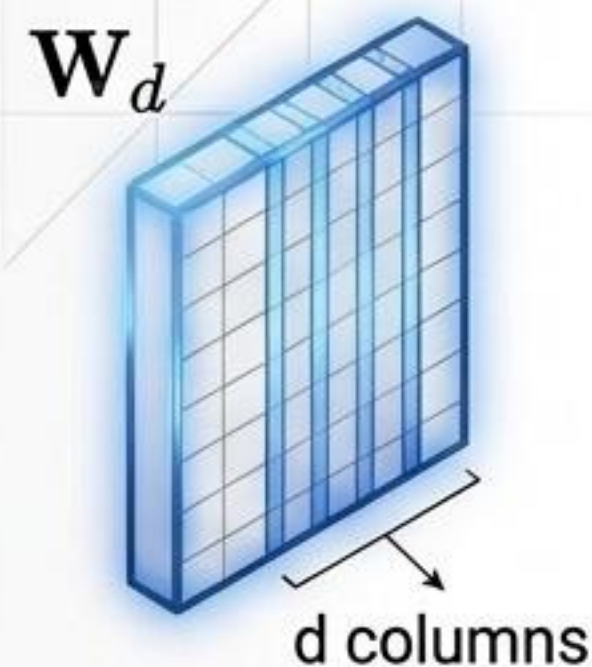
c1 = Vt[0] # Thành phần chính thu 1 (PC1)
c2 = Vt[1] # Thành phần chính thu 2 (PC2)
```

Chiếu dữ liệu xuống d chiều

Sau khi có các thành phần chính (PC), ta có thể giảm kích thước không gian bằng cách chiếu tập dữ liệu lên siêu phẳng tạo bởi d thành phần chính đầu tiên ($d < n$).



Ma trận W_d được định nghĩa là ma trận chứa d cột đầu tiên của V (hay d hàng đầu tiên của V^T chuyển vị).



Phép chiếu: $X_{d-proj} = X \cdot W_d$

Thực hiện phép chiếu bằng NumPy

LOGIC

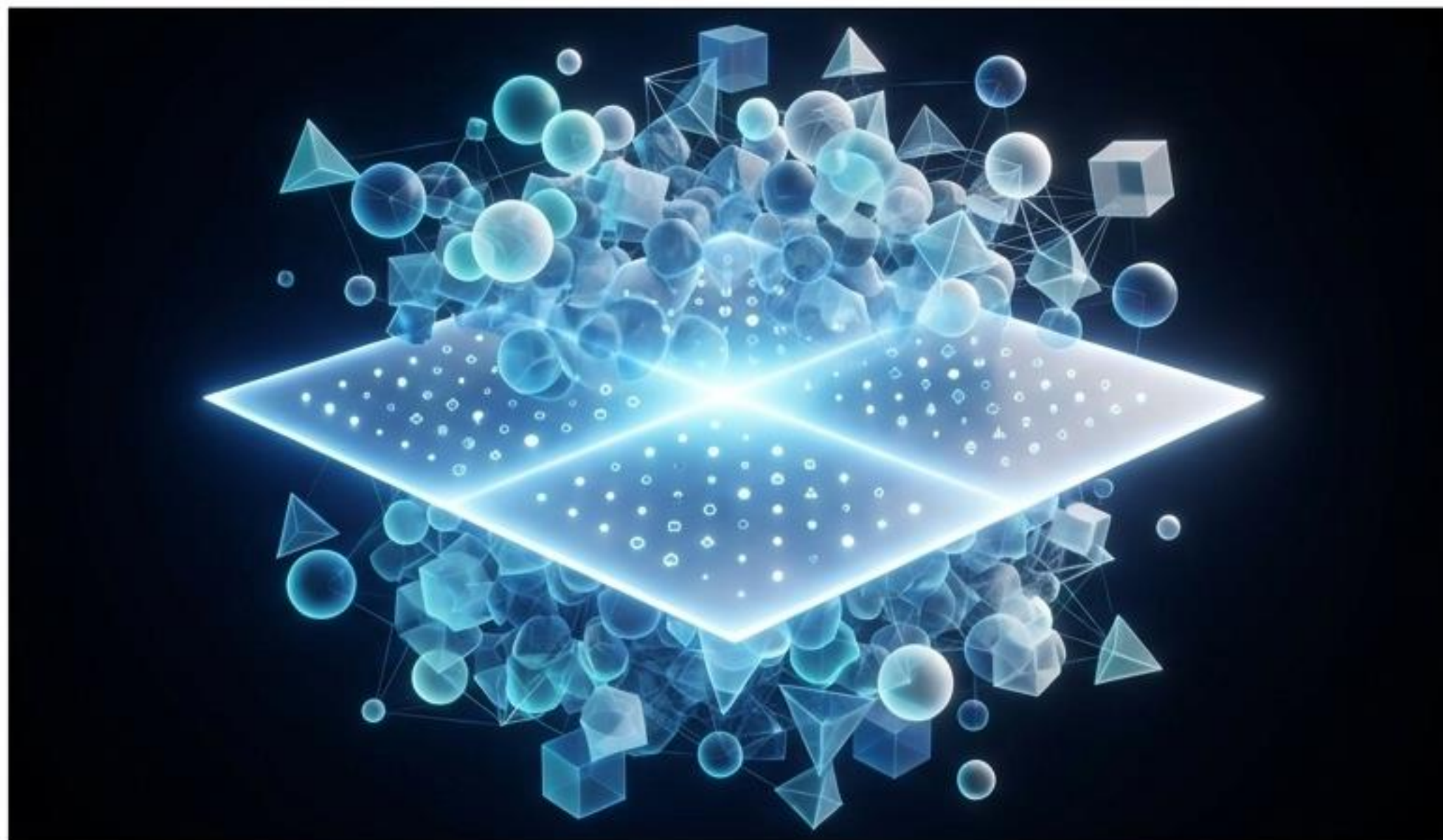


Đoạn mã sau chiếu tập huấn luyện lên **mặt phẳng 2D** định nghĩa bởi hai PC đầu tiên.

SYNTAX

```
# Chọn 2 thành phần chính đầu tiên và chuyển vị  
# Chọn 2 thành phần chính đầu tiên và chuyển vị  
W2 = Vt[:2].T  
  
# Thực hiện phép nhân ma trận (phép chiếu)  
X2D = X_centered @ W2
```

Kết quả: Dữ liệu 2D sau giảm chiều



Bây giờ X2D chính là dữ liệu 2D đã được giảm chiều, bảo toàn được tối đa lượng phương sai có thể từ tập X ban đầu.

Sử dụng Scikit-Learn cho PCA

Trong thực tế, Scikit-Learn cung cấp lớp PCA tự động xử lý SVD và quan trọng nhất là tự động căn giữa dữ liệu. (Không cần trừ đi trung bình thủ công).



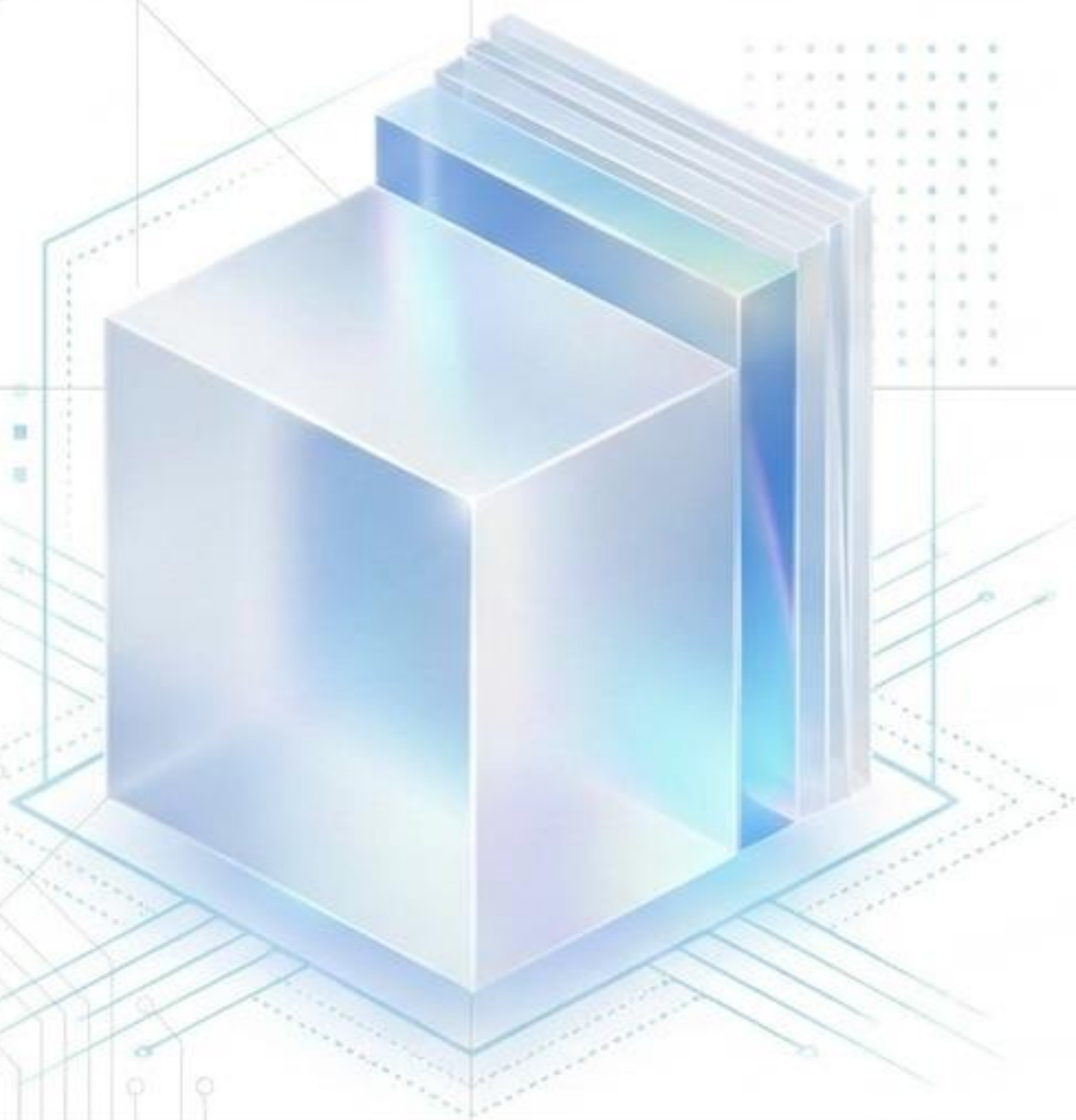
```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X2D = pca.fit_transform(X)

# Lay cac vector thanh phan chinh
print(pca.components_)
```

`pca.components_` chứa các thành phần chính đã tìm thấy.

Tỷ lệ phương sai giải thích (Explained Variance Ratio)



Đây là một chỉ số quan trọng cho biết tỷ lệ phương sai của tập dữ liệu phân bố dọc theo từng thành phần chính. Xem kết quả này qua biến `explained_variance_ratio_`.

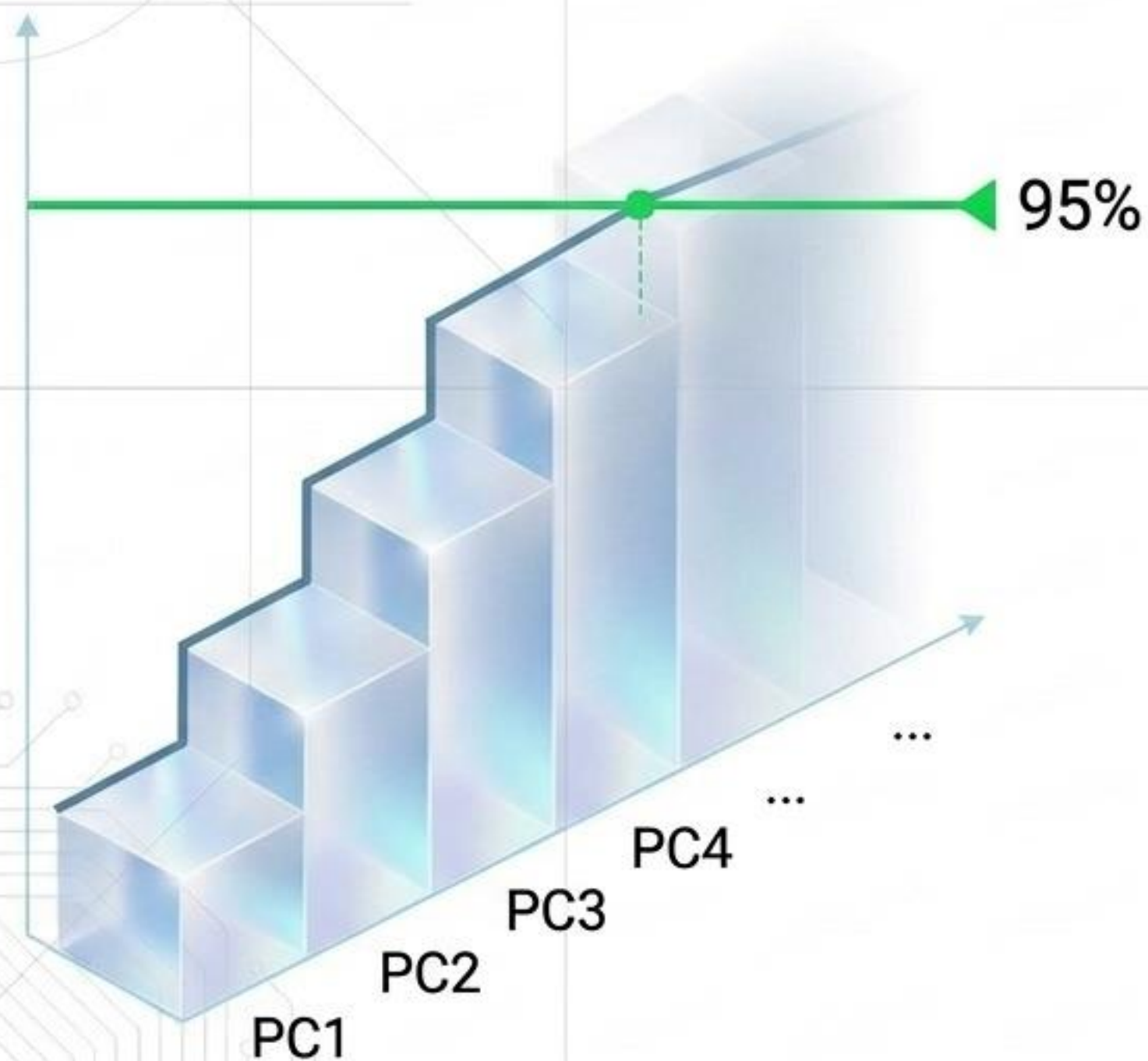
Đánh giá tỷ lệ phương sai giải thích

```
>>> pca.explained_variance_ratio_  
array([0.7578477 , 0.15186921])
```

- Ý nghĩa: $\approx 76\%$ phương sai của dữ liệu nằm dọc theo trục PC1.
 - ◊ $\approx 15\%$ nằm trên trục PC2.
- Tổng cộng: PC1 và PC2 giải thích được $> 90\%$ phương sai.

Kết luận: Điều này ngụ ý rằng các PC tiếp theo (PC3, PC4...) chứa **rất ít thông tin hữu ích** (chỉ còn khoảng 9%).

Chọn số chiều phù hợp như thế nào?

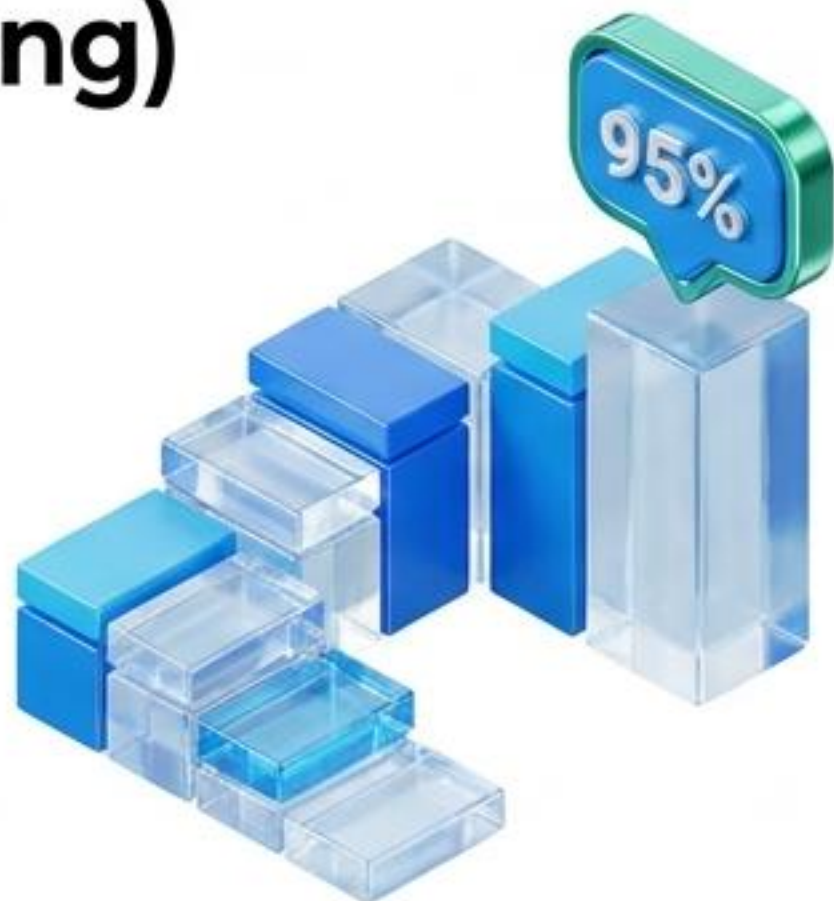


Quy tắc chung:

Trừ khi giảm xuống 2D hoặc 3D để vẽ biểu đồ trực quan, hãy chọn số chiều d sao cho tổng phương sai được giữ lại đủ lớn (ví dụ: 95%).

Nghĩa là, bạn sẽ thêm lần lượt PC1, PC2, PC3... cho đến khi tổng tỷ lệ phương sai giải thích vượt qua mức 95%.

Tìm số chiều để giữ 95% phương sai (Phương pháp thủ công)



Chạy PCA mà không giảm chiều để lấy toàn bộ các PCs. Sau đó, tính toán lũy kế phương sai.

```
from sklearn.datasets import fetch_openml
import numpy as np

mnist = fetch_openml('mnist_784', as_frame=False)
X_train = mnist.data[:60_000]

pca = PCA()
pca.fit(X_train)

cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1
# d bằng 154
```

d = 154

Cách Scikit-Learn giải quyết: Truyền thẳng tỷ lệ phần trăm



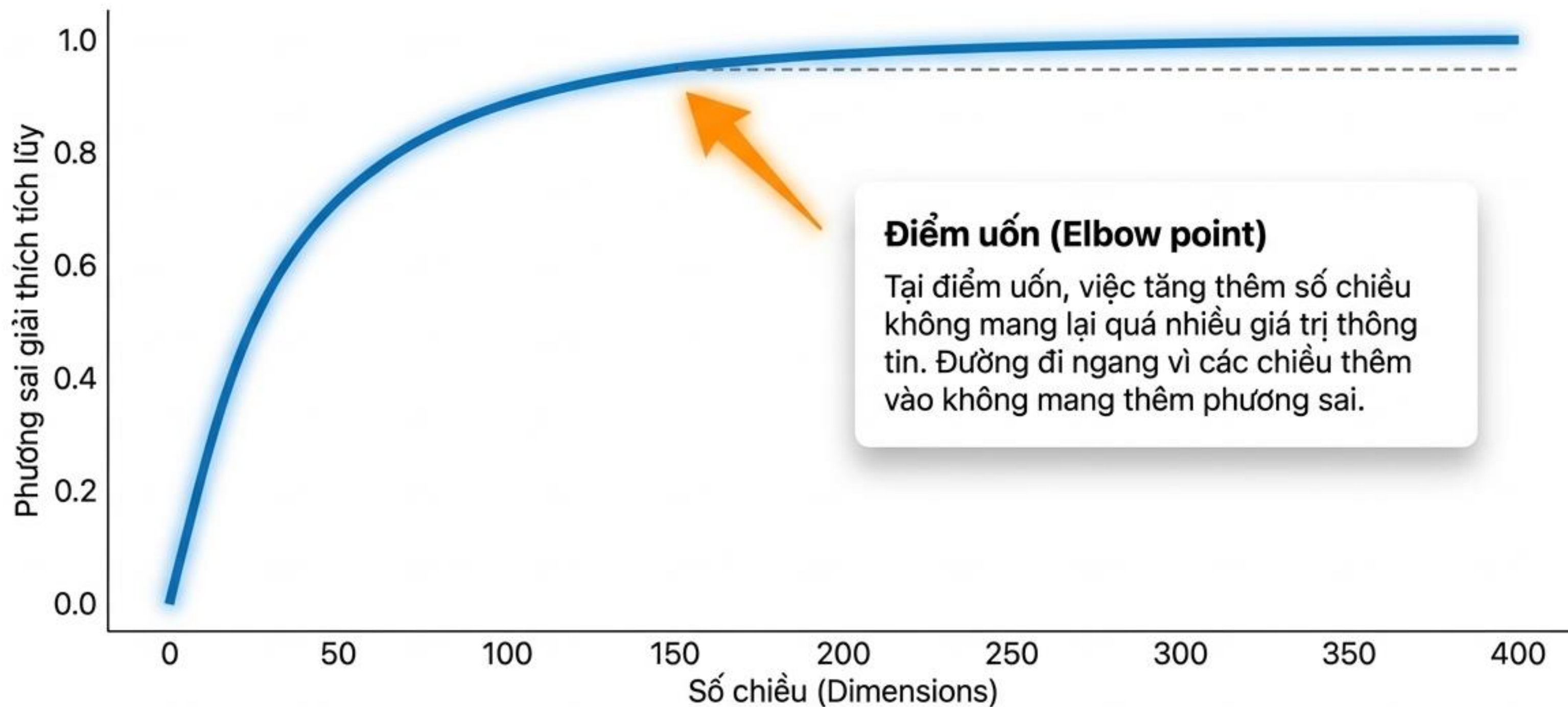
Một cách tốt hơn (và nhàn hơn) là truyền thẳng mức độ phương sai bạn muốn bảo toàn dưới dạng một số thập phân từ 0.0 đến 1.0 cho **n_components**. Scikit-learn sẽ tự tính toán số chiều tối thiểu.

```
# Scikit-learn sẽ tự tính toán số chiều tối thiểu
pca = PCA(n_components=0.95)

X_reduced = pca.fit_transform(X_train)

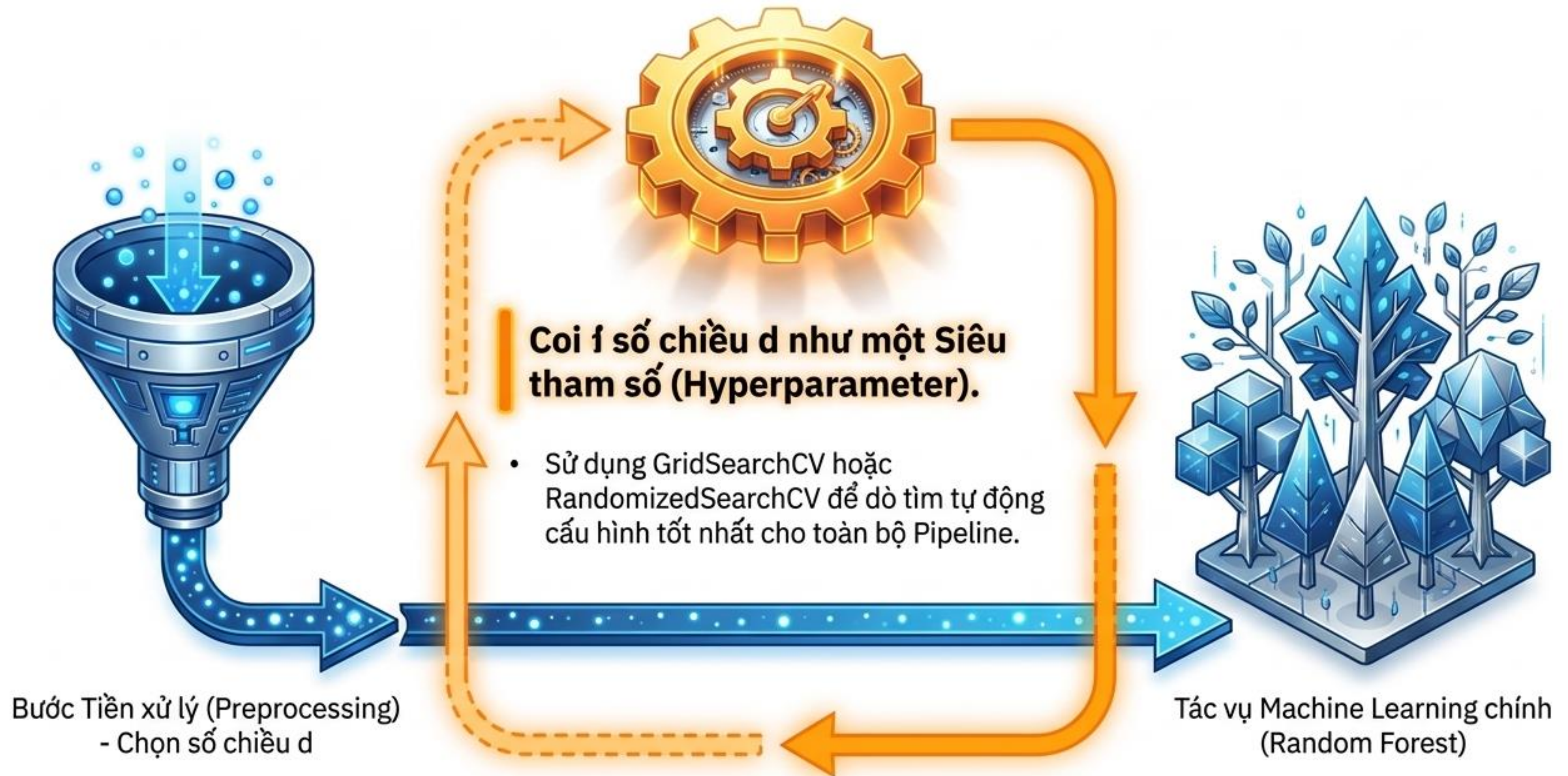
print(pca.n_components_)
# Output: 154
```

Xác định điểm uốn: Vẽ biểu đồ phương sai giải thích



Giúp người phân tích dễ dàng quyết định bằng mắt số chiều cần thiết.

Tối ưu hóa: Điều chỉnh số chiều như một Siêu tham số



Cài đặt RandomizedSearchCV cho Pipeline PCA

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(PCA(random_state=42), RandomForestClassifier(random_state=42))
param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}
rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3, random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])

print(rnd_search.best_params_)
```

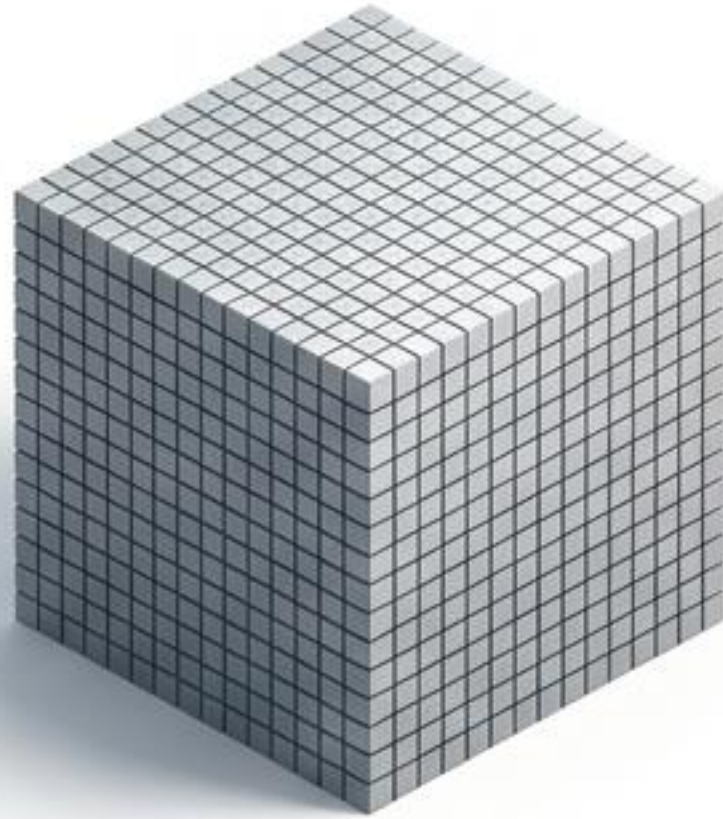


Kết quả bất ngờ!

pca__n_components tối ưu được tìm thấy chỉ là 23 thay vì 154 (khi chạy độc lập).

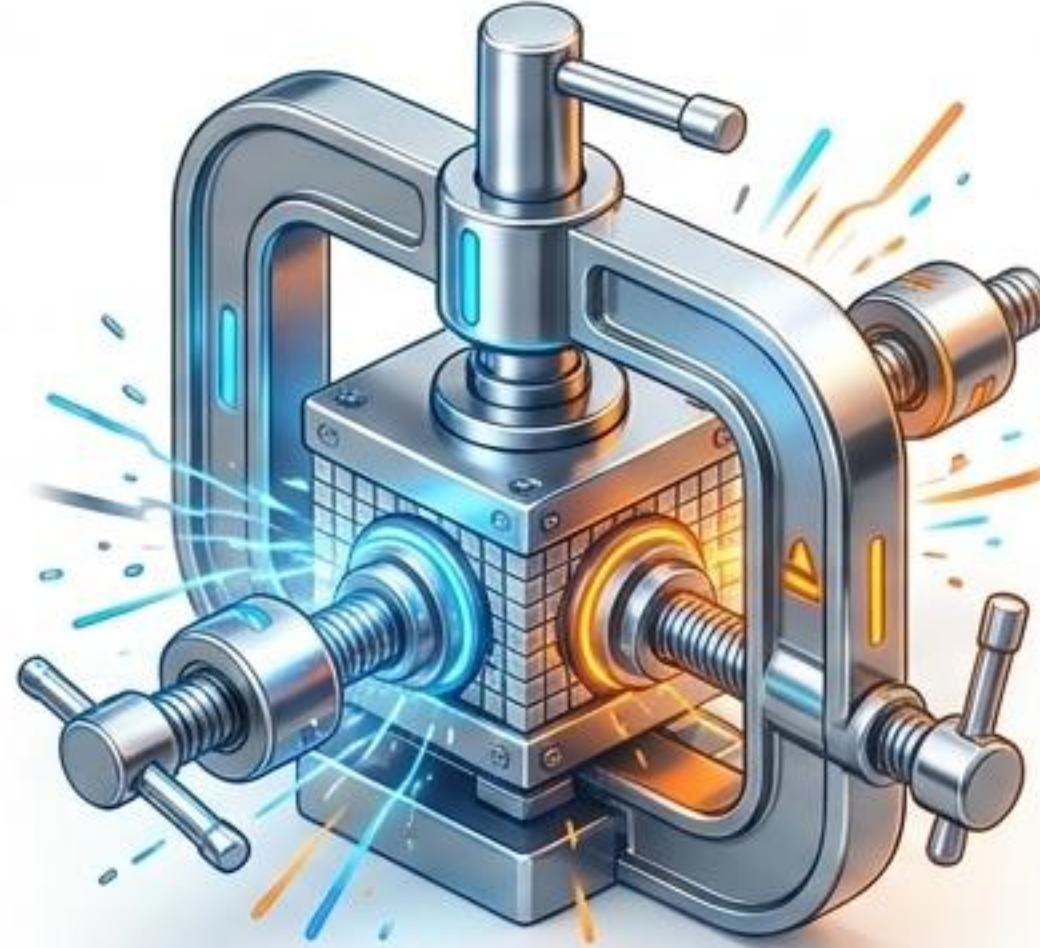
- Mô hình phía sau (Random Forest) hoạt động tốt hơn với dữ liệu được nén chặt hơn.

Bản chất của PCA: Cỗ máy nén dữ liệu



Dữ liệu gốc

784 Đặc trưng (100% kích thước)



Dữ liệu nén (MNIST)

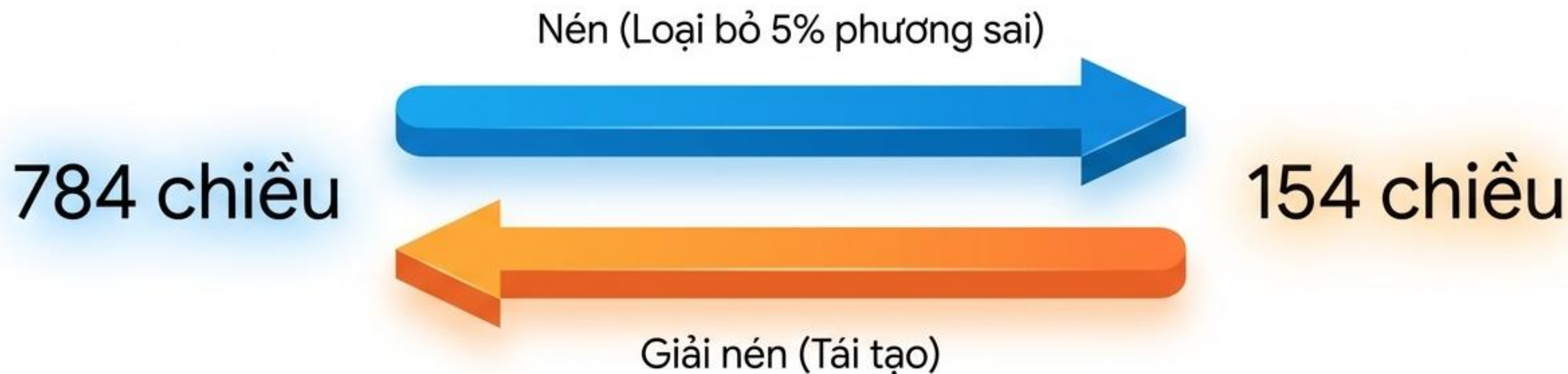
154 Đặc trưng ($\approx 20\%$ kích thước)



Chỉ mất 5% thông tin!

- Tiết kiệm bộ nhớ RAM tuyệt vời.
- Tăng tốc độ thuật toán học máy phía sau vượt trội.

Tái tạo dữ liệu: Nén có hao hụt (Lossy Compression)



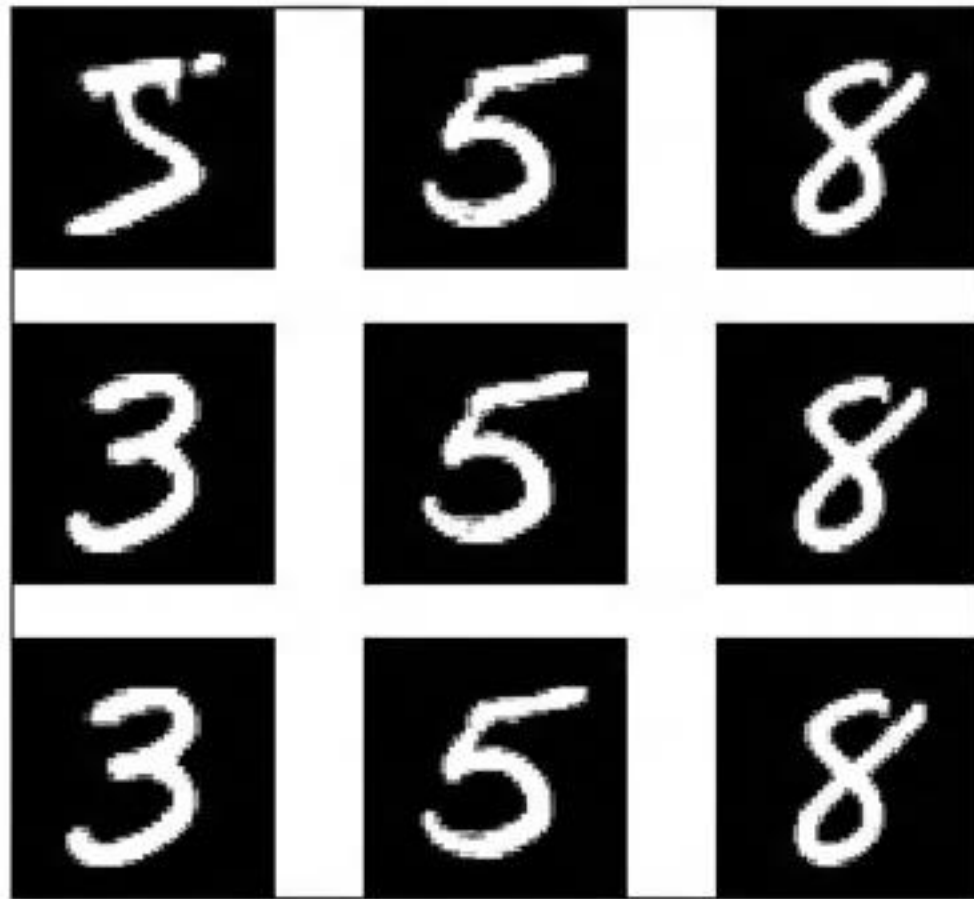
Sai số tái tạo (Reconstruction Error)

Quá trình giải nén sẽ không nhận lại được 100% pixel gốc. Hình ảnh bị mờ nhẹ nhưng phần lớn nội dung vẫn nguyên vẹn.

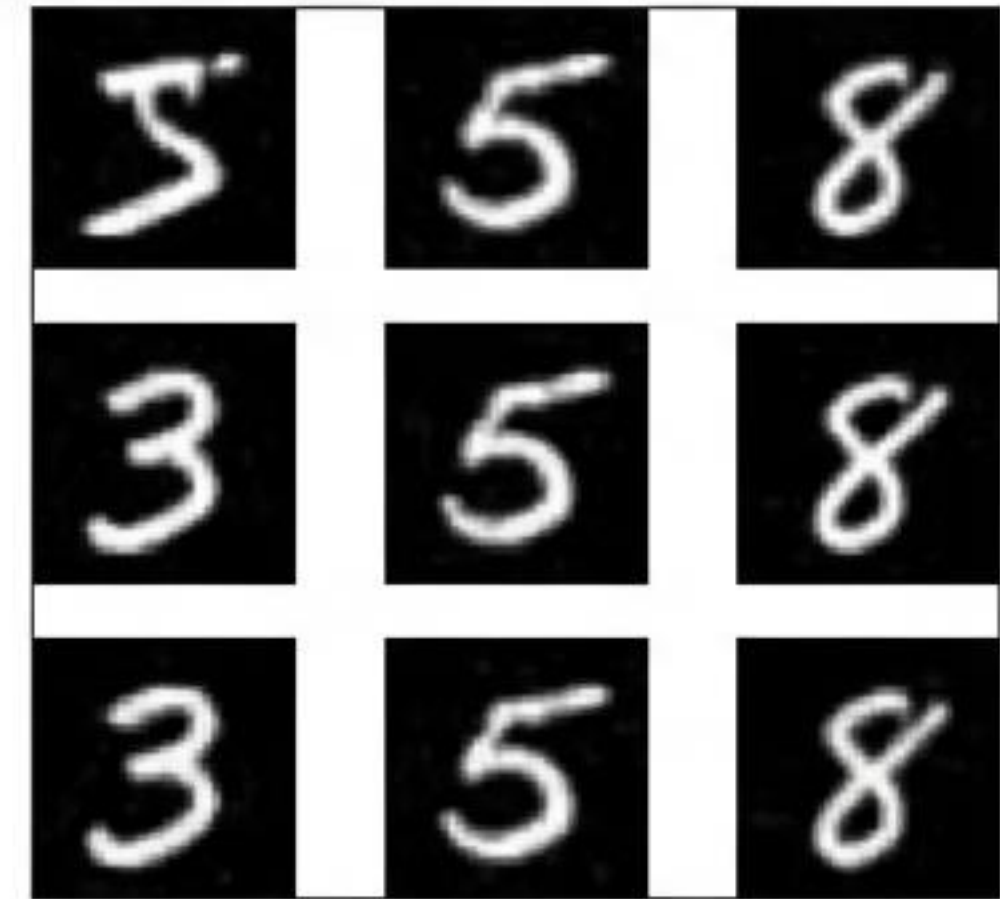
Định nghĩa: Khoảng cách bình phương trung bình giữa Dữ liệu gốc và Dữ liệu giải nén.

Phép biến đổi ngược (inverse_transform)

```
# Sử dụng inverse_transform trên đối tượng PCA  
X_recovered = pca.inverse_transform(X_reduced)
```



Dữ liệu gốc (784 features)



Dữ liệu tái tạo (Từ 154 features)

Mặc dù mờ đi do thiếu 5% phương sai, mắt người (và thuật toán) vẫn dễ dàng nhận diện.

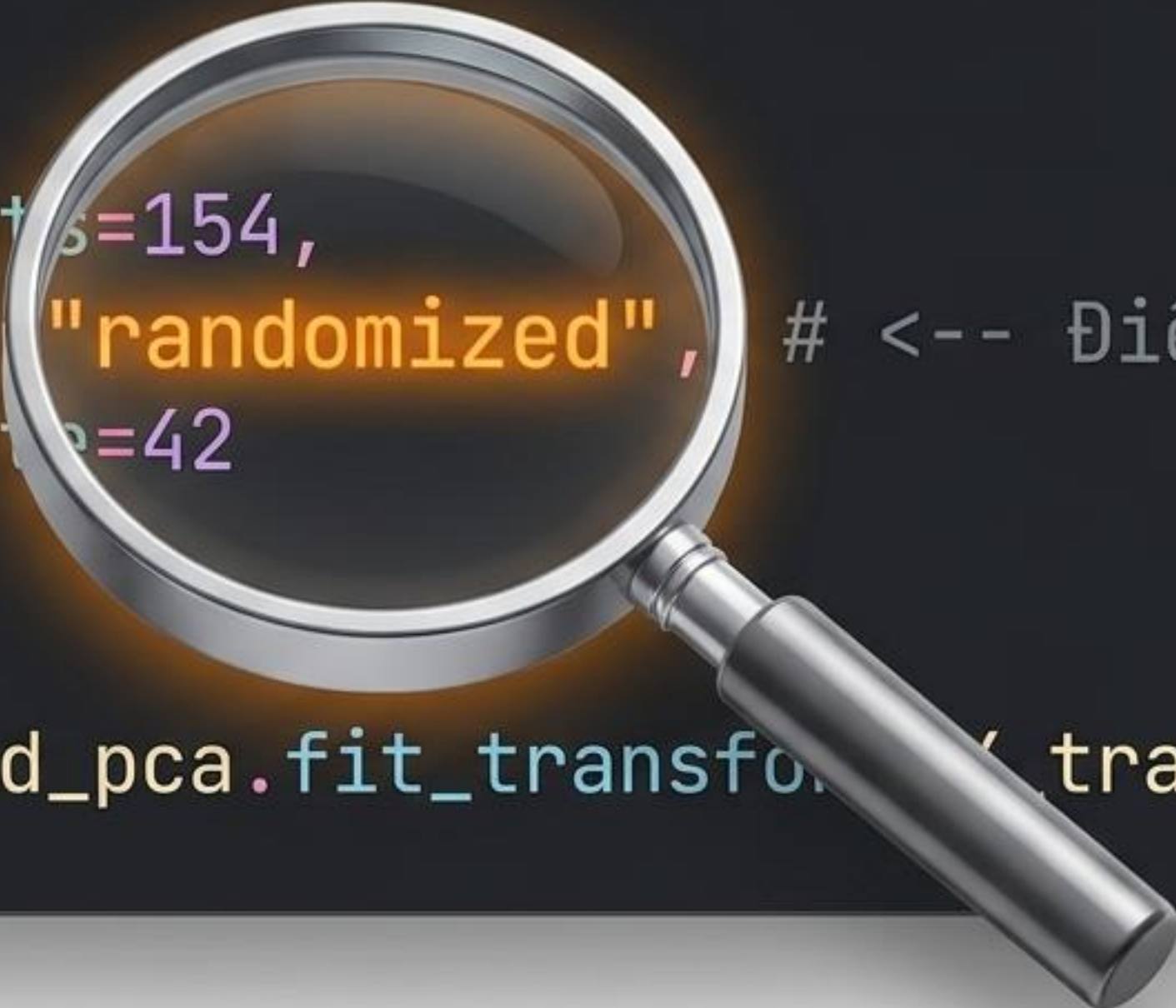
Vượt qua giới hạn tốc độ với Randomized PCA

Thuật toán dùng kỹ thuật ngẫu nhiên hóa để xấp xỉ tìm d thành phần chính đầu tiên một cách nhanh chóng.

SVD Tiêu chuẩn (PCA Đầy đủ)	Randomized PCA
	
$O(m \times n^2) + O(n^3)$	$O(m \times d^2) + O(d^3)$

Nhanh hơn đáng kể khi $d \ll n$ (số chiều giảm xuống nhỏ hơn nhiều so với số chiều ban đầu).

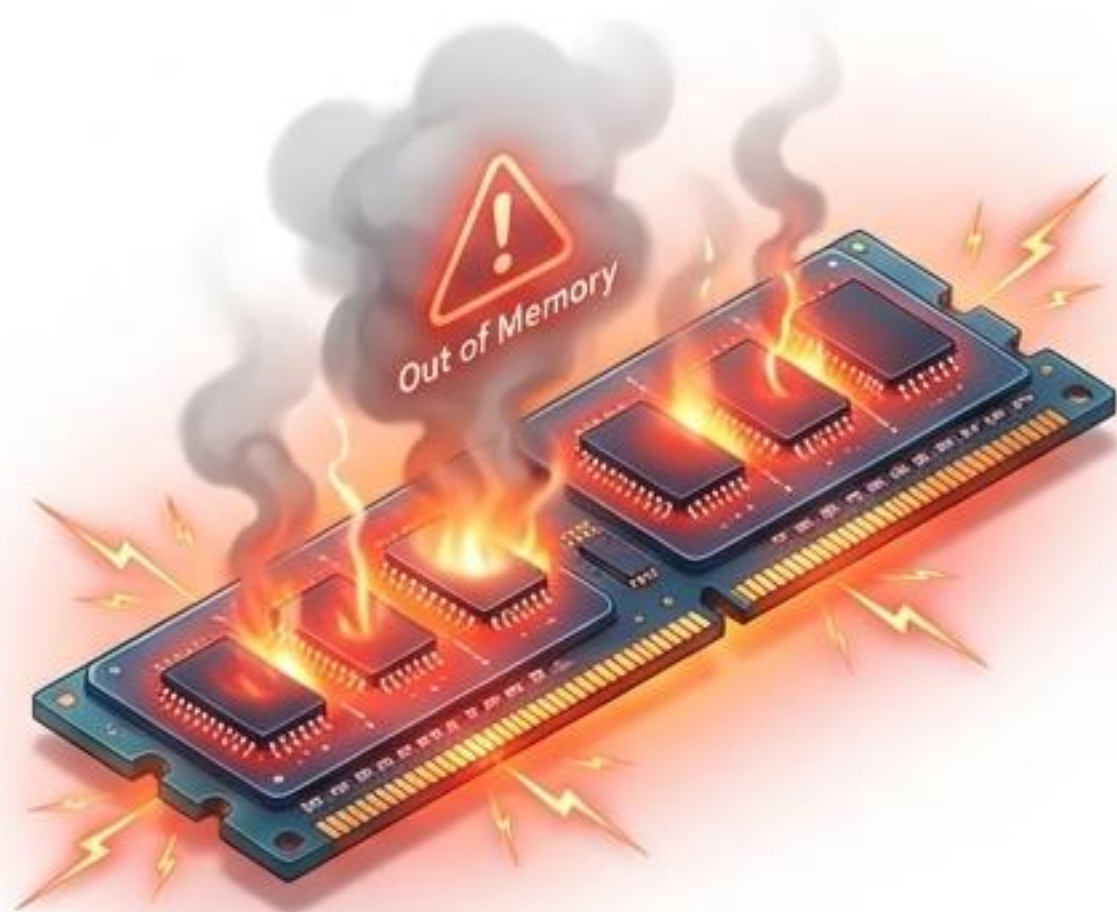
Kích hoạt Randomized PCA trong Scikit-Learn



```
rnd_pca = PCA(  
    n_components=154,  
    svd_solver="randomized", # <-- Điểm mấu chốt  
    random_state=42  
)  
  
X_reduced = rnd_pca.fit_transform(train)
```

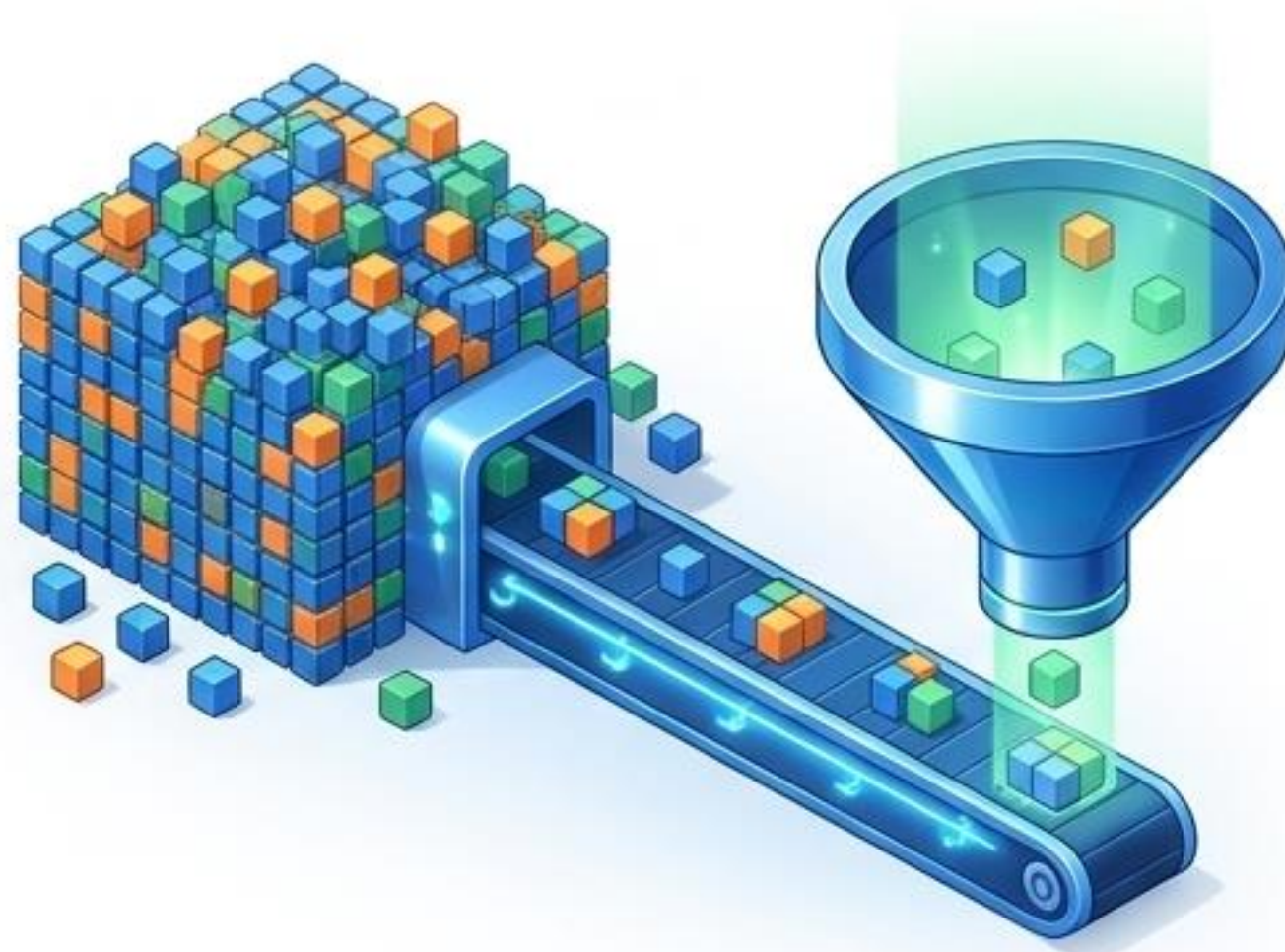
PCA Tăng dần (Incremental PCA): Xử lý dữ liệu khổng lồ

Vấn đề của PCA thông thường



Yêu cầu TOÀN BỘ tập dữ liệu khổng lồ phải nằm gọn trong bộ nhớ (RAM) cùng một lúc.
Rất dễ gây tràn RAM.

Giải pháp: Incremental PCA (IPCA)



Chia nhỏ tập huấn luyện thành các mini-batch
và đẩy vào mô hình từng lô một.

Out-of-core learning

Online Processing

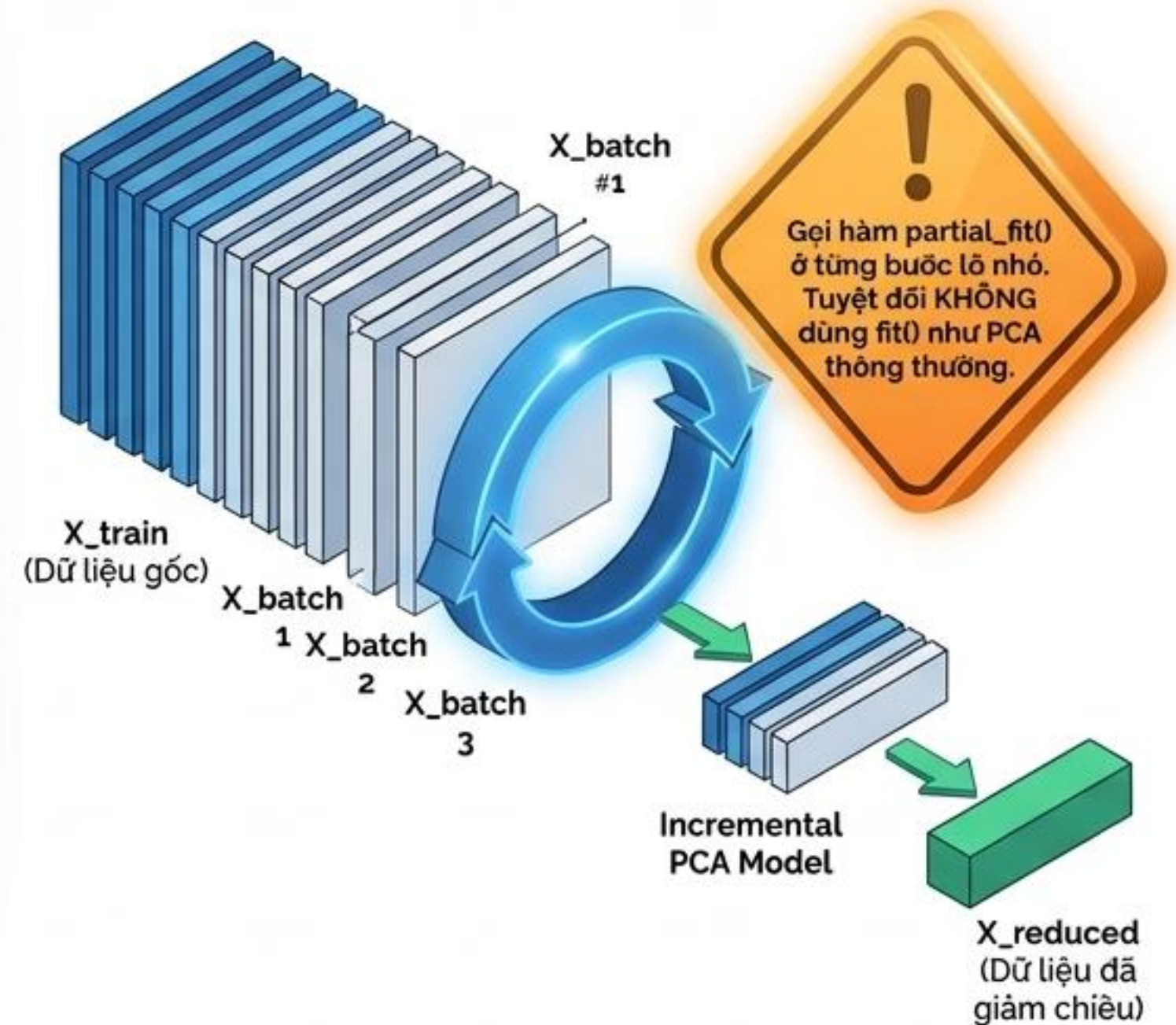
Cài đặt IPCA bằng cách chia mảng (array_split)

```
from sklearn.decomposition import IncrementalPCA
import numpy as np

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)

for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch) # <-- Lưu ý

# transform áp dụng cho toàn bộ
X_reduced = inc_pca.transform(X_train)
```



Tối ưu tối đa: Incremental PCA kết hợp memmap

Tập nhị phân lưu trên đĩa (my_mnist.memmap)

Chỉ tải từng khối batch_size vào RAM khi cần



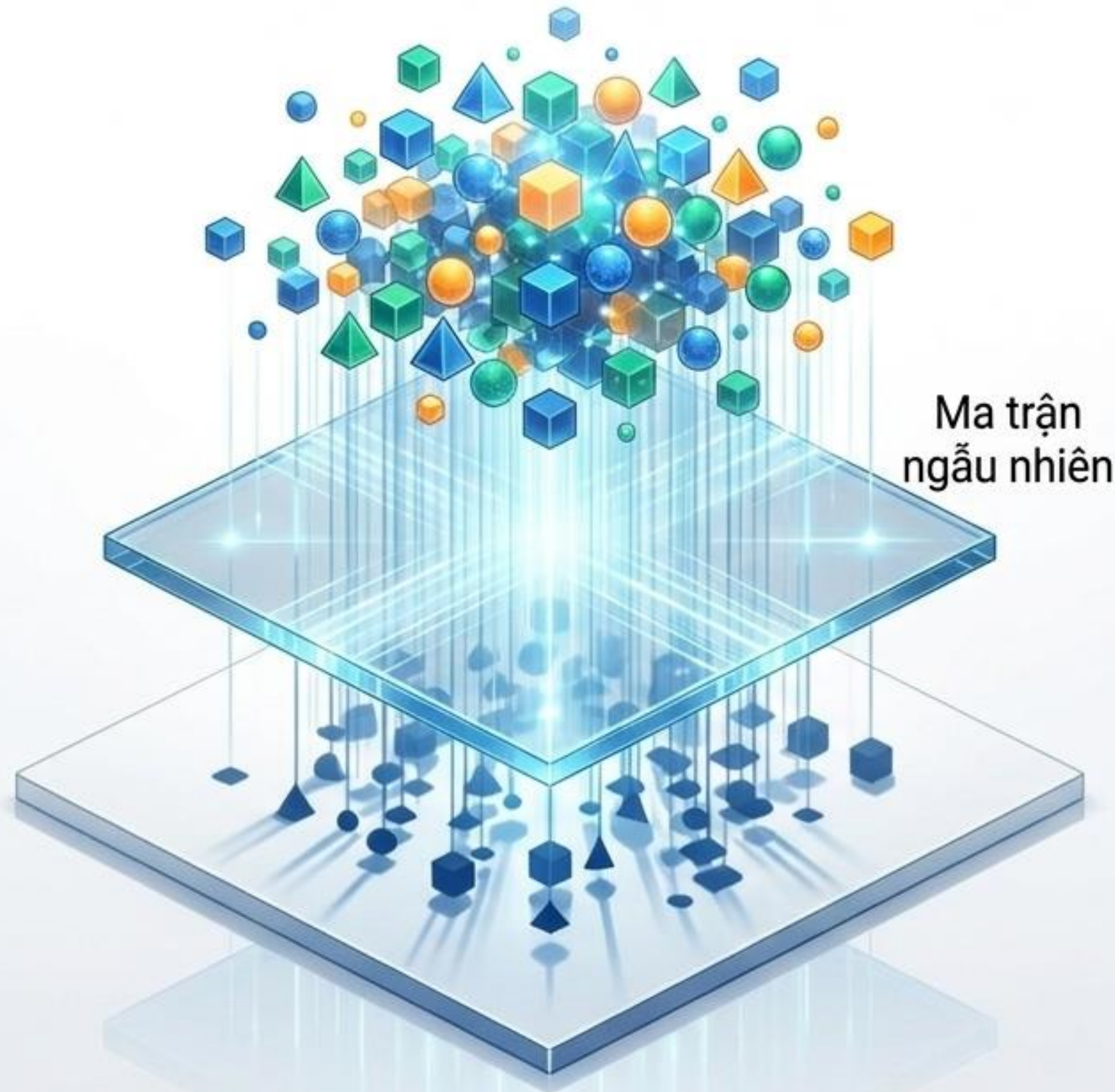
```
# Đọc file đã lưu dưới dạng memmap, mode readonly
X_mmap = np.memmap("my_mnist.memmap", dtype="float32", mode="readonly").reshape(-1, 784)

batch_size = X_mmap.shape[0] // n_batches
inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)

# inc_pca tự động hiểu memmap và lấy từng đoạn theo batch_size
inc_pca.fit(X_mmap)
```

✅ Lúc này có thể gọi `fit(X_mmap)` trực tiếp vì IPCA đã tự động hiểu cấu trúc luồng của memmap.

Phương pháp thay thế: Phép chiếu ngẫu nhiên (Random Projection)



Chiếu dữ liệu xuống **không gian thấp** bằng một **ma trận ngẫu nhiên**.

Kỳ diệu thay: Rất có khả năng **bảo toàn khoảng cách** giữa các điểm dữ liệu.

KHÔNG CẦN HUẤN LUYỆN!

Chỉ phụ thuộc vào hình dạng ma trận ban đầu. Rất nhẹ & cực nhanh cho dữ liệu có hàng chục nghìn features.

Cơ sở Toán học: Định lý Johnson-Lindenstrauss (JL)



Có thể chiếu m điểm từ không gian chiều cao xuống không gian d chiều sao cho khoảng cách bình phương giữa mọi cặp điểm chỉ bị méo mó tối đa là một sai số ϵ .

Ví dụ:

$m = 5.000$ điểm

Dung sai $\epsilon = 0.1$

→ Định lý tính được $d \approx 7300$ chiều là đủ an toàn.

KHÔNG quan tâm đến số chiều gốc n .

(Ngay cả khi $n = 1$ triệu). Định lý phá vỡ rào cản tính toán truyền thống.

Tìm số chiều tối thiểu bảo toàn khoảng cách

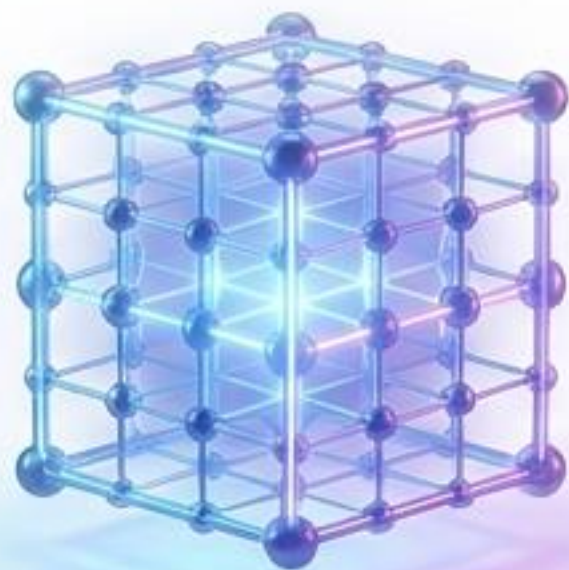
Sử dụng Scikit-Learn để tính toán theo JL lemma.



```
from sklearn.random_projection import  
johnson_lindenstrauss_min_dim  
  
m = 5000 # so truong hop  
eps = 0.1 # dung sai  
  
d = johnson_lindenstrauss_min_dim(m, eps=eps)  
print(d)  
# Output: 7300
```

Phép chiếu ngẫu nhiên thủ công

Tạo một ma trận P lấy mẫu từ phân phối chuẩn và nhân trực tiếp.



```
1 import numpy as np
2
3 n = 20_000 # so chieu ban dau
4 m = 5_000 # so mau
5 d = 7300 # chieu mục tiêu
6
7 # Tao ma tran ngau nhien P
8 P = np.random.randn(d, n) / np.sqrt(d)
9
10 X = np.random.randn(m, n)
11
12 # Thuc hien giam chieu
13 X_reduced = X @ P.T
14
```

Scikit-Learn Random Projections

Dễ dàng hơn với 2 lớp xây dựng sẵn:



```
1 from sklearn.random_projection import GaussianRandomProjection
2 from sklearn.random_projection import SparseRandomProjection
3
4 # 1. Gaussian Random Projection
5 gaussian_rnd_proj = GaussianRandomProjection(eps=0.1, random_state=42)
6 X_reduced = gaussian_rnd_proj.fit_transform(X)
```



```
8 # 2. Sparse Random Projection
9 sparse_rnd_proj = SparseRandomProjection(eps=0.1)
```



Hoạt động tương tự nhưng ma trận chiếu hình thành chủ yếu bằng số 0. **Giúp tiết kiệm RAM cực mạnh** (vd 25MB thay vì 1.2TB).

Nhúng tuyến tính cục bộ (Locally Linear Embedding - LLE)



Phân loại

Là một kỹ thuật **Giảm chiều phi tuyến tính (NLDR)** và thuộc họ **Học đa tạp (Manifold Learning)**. Không sử dụng phép chiếu như PCA.

Cơ chế

1. **Đo lường** cách mỗi điểm liên kết tuyến tính với các **điểm láng giềng gần nhất** của nó.



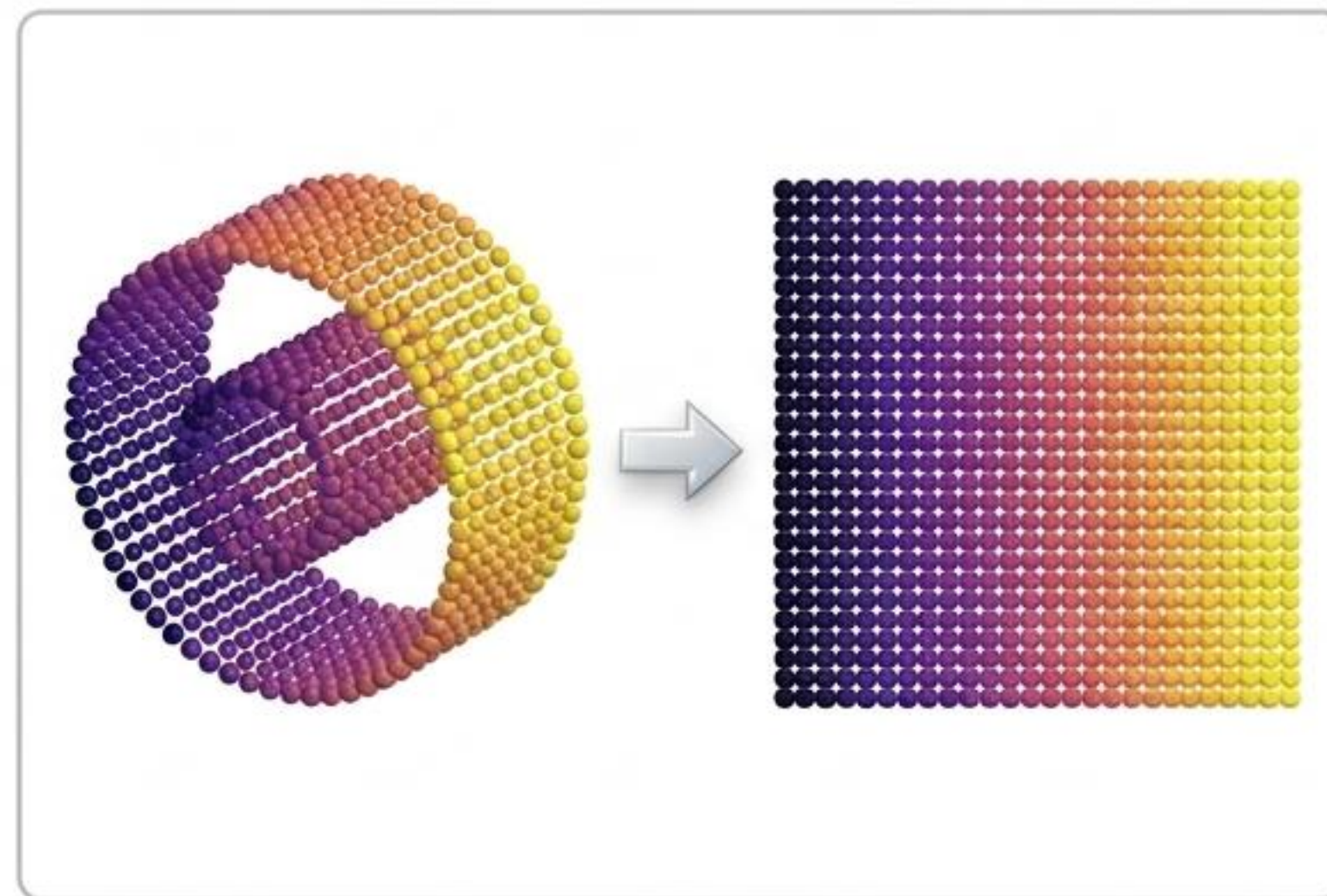
2. **Tìm biểu diễn** chiều thấp sao cho mỗi **liên kết (khoảng cách cục bộ)** này được **bảo toàn tốt nhất**.

Ứng dụng

Tuyệt vời để **mở cuộn** các **đa tạp bị xoắn** (như Swiss roll) nếu dữ liệu **không có quá nhiều nhiễu**.

Sử dụng LocallyLinearEmbedding cho Swiss roll

```
1 from sklearn.datasets import make_swiss_roll
2 from sklearn.manifold import LocallyLinearEmbedding
3
4 # Tao du lieu Swiss roll 3D
5 X_swiss , t = make_swiss_roll(n_samples=1000,
6                               noise=0.2, random_state=42)
7
8 # Giảm xuống 2D bằng LLE, danh gia tren 10
9   lang giống gan nhất
10 lle = LocallyLinearEmbedding(n_components=2,
11                              n_neighbors=10, random_state=42)
12 X_unrolled = lle.fit_transform(X_swiss)
```



(Giả lập Hình 8-10: Dữ liệu Swiss Roll 3D được "mở cuộn" thành một mặt phẳng 2D thành công, duy trì quan hệ giữa các điểm bên cạnh nhau).

Thuật toán LLE: Bước 1 (Mô hình hóa cục bộ)

Đối với mỗi điểm $x(i)$, LLE tìm k láng giềng gần nhất của nó.



Cố gắng **tái tạo điểm** $x(i)$ như một hàm tuyến tính (tổng có trọng số) của k láng giềng đó.

Mục tiêu: Tìm các trọng số $w_{i,j}$ sao cho khoảng cách bình phương giữa $x(i)$ và $\sum w_{i,j}x(j)$ là nhỏ nhất có thể.

Kết thúc bước 1, ta có một ma trận trọng số W phản ánh chính xác cấu trúc hình học **cục bộ của tập dữ liệu trong không gian gốc** (chiều cao).



Thuật toán LLE: Bước 2 (Giảm chiều bảo toàn khoảng cách)

4 Ánh xạ các điểm vào không gian d chiều (với $d < n$).

5 Giữ nguyên ma trận trọng số W đã tìm ở Bước 1.

6 Tìm các vị trí điểm mới (tọa độ mới) trong không gian d chiều, sao cho chúng **vẫn thỏa mãn ma trận trọng số** đó càng gần càng tốt.



Điểm yếu của LLE: Mở rộng kém cho tập dữ liệu rất lớn.

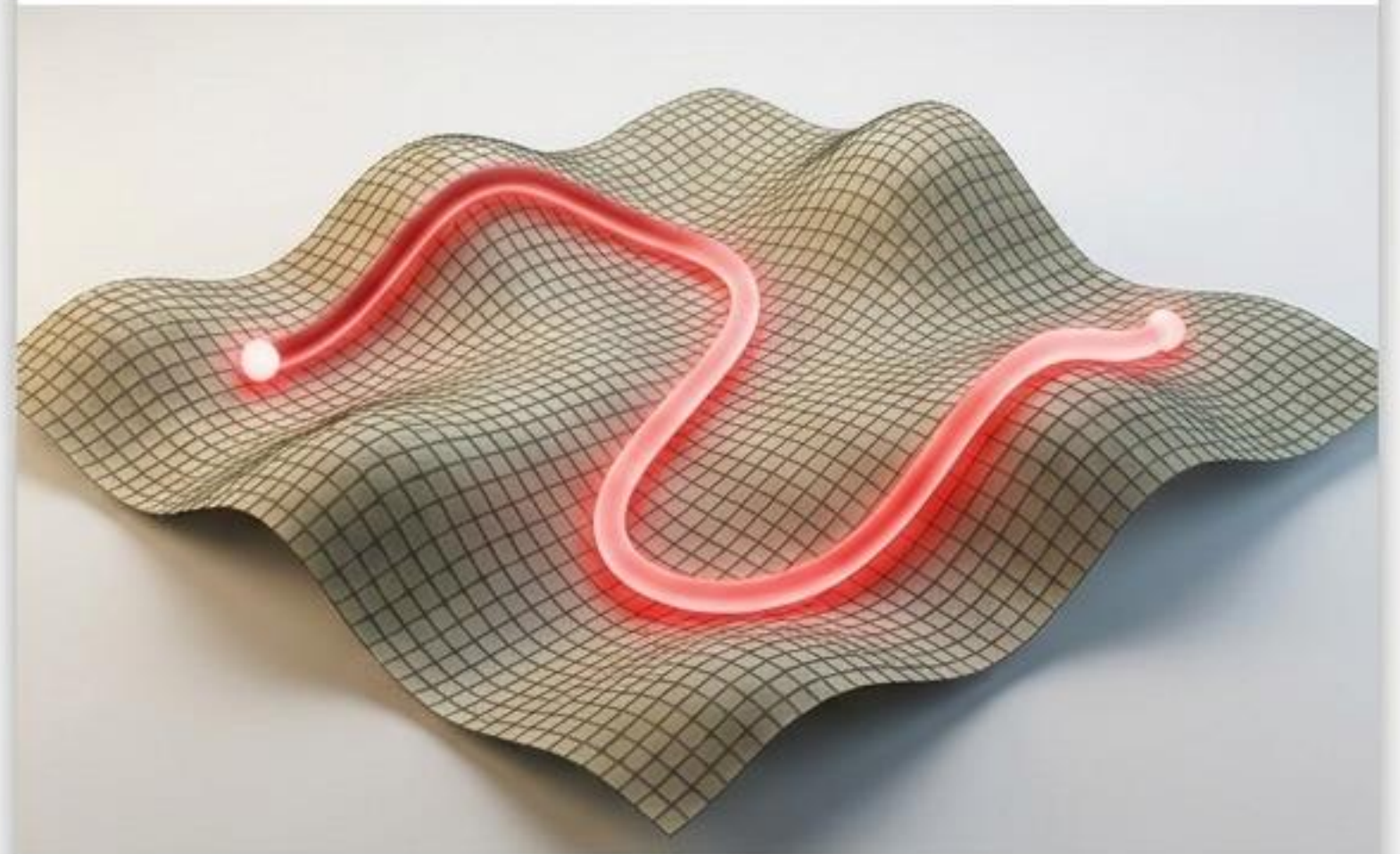
Các kỹ thuật giảm chiều khác: MDS & Isomap

MDS (Multidimensional Scaling)



Cố gắng giảm chiều trong khi bảo toàn **khoảng cách vật lý** giữa các trường hợp như ở không gian gốc. Làm phẳng thay vì mở cuộn.

Isomap



Xây dựng đồ thị nối các láng giềng, sau đó giảm chiều trong khi bảo toàn **khoảng cách trắc địa (geodesic distance)** giữa các điểm. (Trắc địa là đi qua các điểm kết nối, không phải bay đường chim bay). Thích hợp mở cuộn đa tạp.

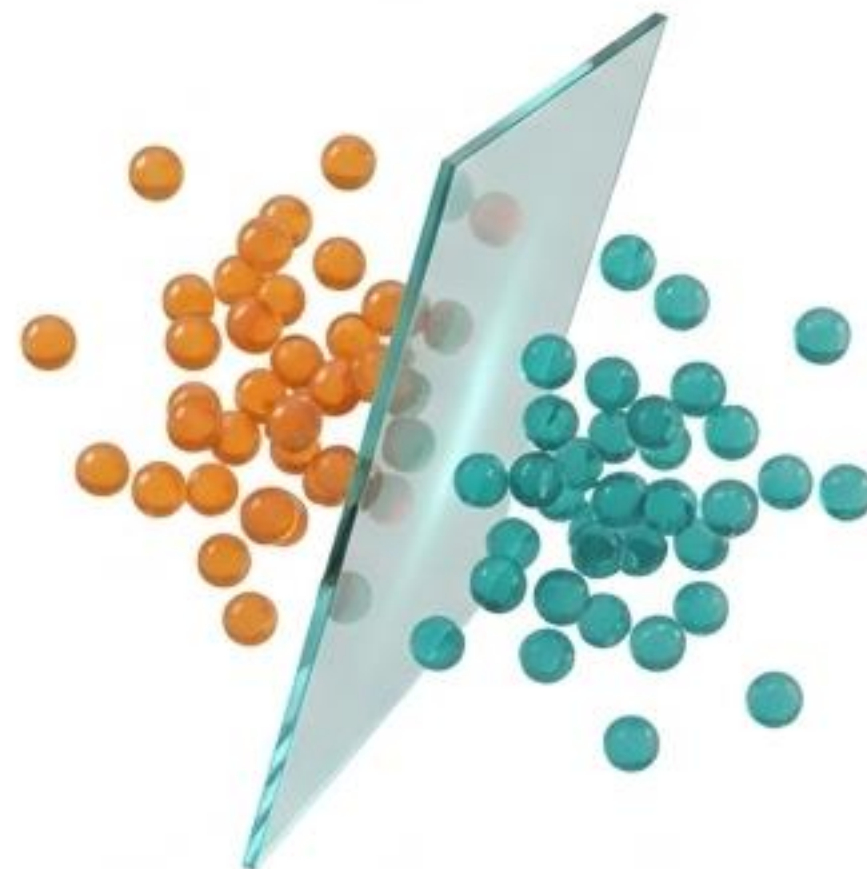
Các kỹ thuật giảm chiều khác: t-SNE & LDA

t-SNE (t-Distributed Stochastic Neighbor Embedding)



Giảm chiều bằng cách giữ các điểm tương đồng ở gần nhau, nhưng đẩy các điểm dị biệt ra thật xa. Đặc biệt tốt để **phát hiện cụm trực quan** (như vẽ bản đồ MNIST 2D).

LDA (Linear Discriminant Analysis)



Một thuật toán **Phân loại** (Classification). Trong quá trình huấn luyện nó sẽ **học trực phân biệt rõ rệt nhất giữa các lớp**. Bạn dùng trục này để chiếu dữ liệu, giúp giữ cho các nhãn cách xa nhau nhất có thể → Tiền xử lý tuyệt vời cho thuật toán phân loại khác.

Tổng kết Chương 8



Khái niệm & Động lực

Giảm số chiều dữ liệu giúp phòng tránh "Lời nguyền số chiều", tăng tốc mô hình và khả năng trực quan hóa.



PCA

Phương pháp Phép chiếu, sử dụng SVD, là kỹ thuật mặc định ưu tiên số một. Nắm vững Incremental PCA (cho dữ liệu lớn) và Randomized PCA.



Random Projection

Vô địch về tốc độ và tiết kiệm RAM cho dữ liệu siêu lớn mà không cần huấn luyện.



Manifold Learning (LLE, Isomap...)

Có khả năng "mở cuộn" các hình khối xoắn ốc phức tạp, thích hợp cho dữ liệu có cấu trúc đa tạp phi tuyến tính.



CẢM ƠN CÁC BẠN ĐÃ LẮNG NGHE!

Q & A